Designing a Dynamic Policy Enforcer Using Normative Languages for Microservice-based Middleware

Version of 22nd May 2026

Niels Arts

Designing a Dynamic Policy Enforcer Using Normative Languages for Microservice-based Middleware

THESIS

submitted in partial fulfilment of the
requirements for the degree of

MASTER OF SCIENCE

in

SOFTWARE ENGINEERING

by

Niels Arts

under the supervision of

Dr. Ana Oprescu (CCI, UvA)

May 2026

Master Software Engineering
Informatics Institute
FNWI, University of Amsterdam
Amsterdam, The Netherlands

Complex Cyber-Infrastructures
Science Park 900
1098 XH Amsterdam
The Netherlands

Thesis Committee

Examiner (chair):	Dr. Ana Oprescu (CCI, UvA)
Second Reviewer:	Dr. Thomas L. van Binsbergen (CCI, UvA)
Daily Supervisor:	Alexandros Koufakis (CCI, UvA)

Abstract

Cross-organisational data sharing through Digital Data Marketplaces (DDMs) requires policy enforcement that is both formally grounded and adaptive to runtime change. Existing DDM middleware platforms, such as DYNAMOS developed at the University of Amsterdam, rely on procedural policy enforcers that lack normative reasoning, support only pre-access decisions, and conflate enforcement responsibilities with orchestration logic. These shortcomings undermine data sovereignty guarantees and limit compliance with regulations such as the GDPR and the Data Governance Act (DGA) in dynamic, multi-party collaboration settings.

This thesis addresses the question: *How can a policy enforcer functionality be integrated in a microservice-based DDM middleware to ensure compliance with mutable policies?* Following a Design Science methodology, the work is structured around three sub-questions covering architectural integration, dynamic agreement updates, and correctness of the resulting integration.

The proposed Policy Enforcer decomposes into six internal components aligned with XACML data-flow roles and UCON+ continuous authorisation, embedding an eFLINT normative reasoner via a modular adapter that allows both the reasoning engine and the policy store back-end to be replaced independently. A three-layer eFLINT specification, comprising an interface policy, agreement rules, and request facts, provides a stable query schema while allowing runtime-mutable agreements to be swapped without modifying enforcer code. When an agreement changes, the Policy Enforcer drives validation, identifies affected in-flight sessions, and re-evaluates each session against the new policy, producing per-session decisions that result in no-op, recomposition, or revocation outcomes.

Correctness is established through differential testing against the eFLINT reasoner as oracle, organised into four orientations: request approval, continuous authorisation, dynamic policy updates, and audit records. Across seven request-approval scenarios the implementation produces decisions structurally equivalent to the oracle projection while additionally backing each decision with a formal derivation tree. The prototype integrates into DYNAMOS without breaking existing public interfaces, demonstrating that normative, continuously authorising, and auditable policy enforcement is feasible within a microservice-based DDM middleware.

Contents

List of Figures

List of Listings

List of Tables

Acknowledgements

I would like to express my gratitude to the people who made this thesis possible.

First and foremost, I want to thank my academic supervisor, Dr. Ana Maria Oprescu, for giving me the opportunity to work on my master's thesis within the DYNAMOS project at CCI. Given her many responsibilities both within the master's programme and across CCI, I am truly grateful that she still found the time to provide meaningful insights and guidance throughout this project.

I am equally grateful to my daily supervisor for the considerable time he dedicated to guiding me through this project. Having him as a peer to brainstorm with during our regular meetings was invaluable, and I could not have navigated this work without his support.

I also want to thank my partner, Louise Terlouw, for her loving support throughout my master's. This step in my career would not have been possible without her encouragement. She believed in me whenever I doubted myself, and her patience made all the difference.

Finally, I want to thank my fellow students Max Veerhoek and Thom Pheijffer. Having such bright and sharp people around, and someone to grab drinks with, made this journey a lot more enjoyable.

Chapter 1

Introduction

Our society increasingly runs on data. Businesses rely on data to guide strategic decision-making, while academic and public institutions leverage data to support scientific and societal goals. To improve decision-making and facilitate research, the size and diversity of datasets are growing, often through combining datasets across organisational boundaries and infrastructures. Moreover, federated machine learning has emerged as a key trend requiring collaboration across organisational and infrastructural boundaries. This growing demand for large-scale data sharing introduces significant technical and legal complexity. To manage this complexity, Digital Data Marketplaces (DDMs) are being developed, enabling organisations to exchange data and computational resources across trust boundaries.

Governing these cross-organisational exchanges is challenging. Systematic reviews of data market design identify data governance and data sovereignty as central problems. Once consumed, data can be duplicated and redistributed beyond the original provider's control, and no single actor has authority over its use across organisational boundaries [5]. Data owners fear loss of control and worry that sharing may harm their business interests; compounding this, the absence of standardised legal and contractual frameworks creates considerable uncertainty in practice [19].

Within the University of Amsterdam, the Complex Cyber Infrastructure (CCI) group developed DYNAMOS, a DDM middleware that orchestrates distributed microservices and coordinates data flows to facilitate data sharing operations [26]. Additionally, the Multiscale Networked Systems (MNS) group has developed BRANE, which is a Framework for Programmable Orchestration of Multi-Site Applications [27].

Recent literature highlights that effective data sharing across organisational boundaries depends not only on technical orchestration but also on the ability to verifiably enforce policies that protect data sovereignty, comply with regulations (such as the GDPR and the Data Governance Act (DGA)), and adapt to evolving agreements among consortium members [5, 11, 18]. Digital Data Marketplaces (DDMs) must manage dynamic, granular access control in environments where policies may change at runtime and no single party has full authority over data use. Middleware supporting such exchanges must therefore satisfy demanding non-functional requirements: robust security and privacy mechanisms, compliance with regulations such as the General Data Protec-

tion Regulation (GDPR) and the DGA, adaptability to evolving legal requirements, and granular access control [12]. Existing policy languages such as Open Digital Rights Language (ODRL), while widely used for expressing data usage policies, have demonstrated limitations when applied to rigorous regulation of data-sharing infrastructures [17], motivating the exploration of formally grounded normative specification approaches. In this context, middleware platforms such as DYNAMOS [26] motivate the need for robust, formally grounded mechanisms that can automatically enforce and update access and usage policies during ongoing collaborations.

To fill the identified gap in dynamic policy enforcement in DDM middleware, this thesis is driven by the following research question:

> How can a policy enforcer functionality be integrated into a microservice-based DDM middleware to ensure compliance with mutable policies?

To systematically address this primary research question, we break it down into three sub-research questions. Following the classification of research questions by Easterbrook et al. [6], this thesis combines design-oriented questions (focused on how to realise an effective policy-enforcer integration) with an empirical descriptive question (focused on the correctness of the integration):

- **SRQ1: Architectural Integration** — What architectural design enables effective integration of a policy enforcer within dynamic microservice middleware? It is addressed through a targeted literature review and case study analysis of BRANE and DYNAMOS, culminating in an exploratory architectural design presented in chapter 5.
- **SRQ2: Dynamic Agreement Updates** — How can policy changes during runtime dynamically alter the execution of data sharing operations? It is addressed through prototyping and system trace analysis of the policy-update pathway within DYNAMOS, as described in chapter 6.
- **SRQ3: Correctness Evaluation** — To what extent does the implemented policy enforcer correctly enforce normative policies across the defined evaluation scenarios? It is addressed through oracle-based differential testing of the Policy Enforcer against the unmodified eFLINT reasoner across a predefined set of evaluation scenarios, reported in chapter 7.

To address this combination of design-oriented and empirical descriptive questions, this thesis applies a design science methodology. The design-oriented questions (SRQ1 and SRQ2) are addressed by deriving requirements from literature and the DYNAMOS context, and by translating these into an architecture for policy enforcement and runtime policy updates. The descriptive question (SRQ3) is addressed through differential correctness testing of the implemented prototype against the normative eFLINT specification, covering the defined evaluation scenarios while considering threats to validity.

This thesis is organised as follows. chapter 2 introduces the technical and conceptual foundations. chapter 5 presents the architecture. chapter 6 describes the implementation. chapter 7 reports evaluation outcomes. chapter 3 positions this work against existing literature and compares approaches and results. chapter 9 concludes with contributions and future work. chapter 8 interprets the findings.

Chapter 2

Background

This chapter introduces the foundational concepts and systems upon which this thesis builds. We first describe the DYNAMOS middleware platform (section 2.1). We then introduce access control models and the eXtensible Access Control Markup Language (XACML) architecture (section 2.2), followed by normative languages and the eFLINT Domain-Specific Language (DSL) (section 2.3). Next, we discuss usage control models that extend traditional access control (section 2.4). Finally, we describe the BRANE framework, whose integration of eFLINT-based policy reasoning has informed the design decisions in this thesis (section 2.5).

2.1 DYNAMOS: Dynamically Adaptive Microservice-based Middleware

The DYNAMOS project, introduced by Stutterheim [25, 26], is a dynamically adaptive microservice-based system for data exchange developed within the Complex Cyber Infrastructure (CCI) group at the University of Amsterdam. Inspired by how traditional operating systems abstract hardware complexities into standardised core functions, DYNAMOS abstracts different data exchange patterns into a unified set of core data exchange microservices that adhere to agreed-upon policies. The platform employs Monitor–Analyse–Plan–Execute over shared Knowledge (MAPE-K) control loops [1] for self-adaptation, sidecars for communication abstraction, protocol buffers for strict interface definitions, and ephemeral single-use jobs for improved security [26].

However, the policy enforcement component in DYNAMOS was mocked: it simulates request validation and provides archetype information, but does not perform actual normative reasoning. This design allows DYNAMOS to serve as an experimental platform for investigating the interaction between policy-driven decisions and middleware orchestration, while leaving the integration of a genuine policy reasoner as an open challenge, one that this thesis addresses.

2.2 Access Control and the XACML Architecture

Attribute-Based Access Control

In traditional access control models such as Role-Based Access Control (RBAC) [21] and Attribute-Based Access Control (ABAC) [15], an access request identifies the actor making the request, the resource on which the action is to be performed, and the type of action to be performed (e.g., read or write). ABAC generalises earlier models by basing access decisions on attributes of the subject, object, and environment, making it flexible and fine-grained. Standards such as XACML and ODRL are frequently applied to express and enforce such policies [3].

Despite its strengths, ABAC has notable shortcomings in the context of Digital Data Marketplaces (DDMs) and GDPR compliance. First, it provides no native support for *obligations*, actions a subject must perform to gain or sustain access [22]. Second, it handles *authorisation* (the technical concept of granting access) but not *permission* (the legal concept of lawfulness), a distinction emphasised by Binsbergen et al. [3]. Third, enforcement is a one-time, pre-access decision: once access is granted, there is no further control over how the resource is used, and no distinction between ex-ante and ex-post enforcement [3]. These limitations motivate the introduction of normative languages (section 2.3) and usage control models (section 2.4).

The XACML Architecture

The XACML standard defines a widely adopted reference architecture for structuring access and usage control systems [10]. As illustrated in fig. 2.1, it separates policy enforcement into five technical roles:

- The **Policy Administration Point (PAP)** stores policies and determines which policies are applicable to which requests. An administrator registers policies in the PAP by associating them with targets.
- The **Policy Enforcement Point (PEP)** is a control mechanism that intercepts an actor's action on a resource and conditionally prevents its execution. The action is admitted only if the Policy Decision Point (PDP) permits the request.
- The **PDP** evaluates the access request against the applicable policies and returns a permit or deny decision. It receives the set of relevant policies from the PAP and, together with additional information provided by Policy Information Points (PIPs), uses them to make its decision.
- **PIPs** provide additional policy information, such as roles, attributes, and environmental context, that the PDP requires for its evaluation. The information available depends on the specific access control model in use.
- The **Context Handler (CH)** converts decision requests from the PEP's native format into the XACML canonical form, coordinates with PIPs to add attribute values to the request context, and converts the PDP's authorisation decision back into the PEP's native response format.

These technical roles define a conceptual framework for assigning policy enforcement responsibilities among different system components. This framework is technology-

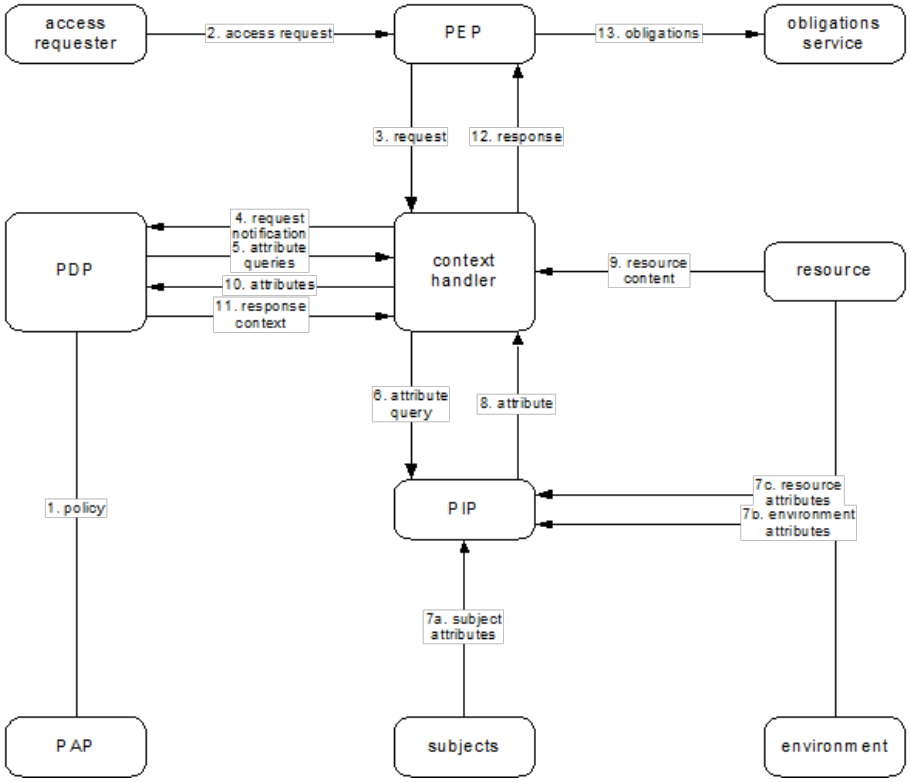

Figure 2.1: Data flow diagram of the XACML model with major actors [10]

neutral, which makes it applicable beyond pure access control [10]. In chapter 5, we reuse these roles to describe how policy enforcement is distributed across DYNAMOS components.

2.3 Normative Languages and eFLINT

The shortcomings of ABAC identified in section 2.2, the lack of obligation support, the absence of legal reasoning, and one-shot enforcement, show that technical access control alone is insufficient for DDM and GDPR settings. Normative languages address this gap by providing formal means to express permissions, obligations, and prohibitions in machine-readable form, bridging the divide between legal agreements and executable policy specifications.

The legal-theoretic foundation for normative languages is provided by Hohfeld's framework of fundamental legal conceptions [14], which underpins eFLINT's normative type system. As shown by van Binsbergen et al. [29], eFLINT specifications reason in an action-oriented way about four central normative notions: permissions, prohibitions, obligations, and duties. Importantly, Binsbergen et al. [3] distinguishes *authorisation*, the technical term related to access control, from *permission*, the legal concept of lawfulness.

This distinction underpins the integration of normative reasoning with enforcement architectures such as XACML.

eFLINT, introduced by van Binsbergen et al. [29], is a DSL for executable norm specifications. Its theoretical foundations lie in transition systems and Hohfeld's framework. The language is action-oriented: actions may be permitted or obliged and can have effects that modify the legal positions of one or more actors. An eFLINT program consists of a *normative specification* (fact-type, duty-type, event-type, and action-type declarations) and a *scenario description* (triggers and queries). The resulting specifications are executable and support automatic case assessment, simulation, and monitoring of running systems.

In subsequent work, Binsbergen et al. [3] demonstrate how eFLINT-based normative reasoning can automate GDPR-based access and usage control in distributed systems. They propose extending the XACML architecture (introduced in section 2.2) to integrate an eFLINT-based PDP, separating case-generic GDPR rules (encoded as eFLINT specifications) from case-specific qualifications (supplied by privacy experts at runtime). This two-layer design promotes transparency, adaptability, and explainable decision-making through the structure of the reasoning process. This thesis builds on that integration approach.

2.4 Usage Control

The UCON Model

As illustrated in fig. 2.2, usage control generalises access control along five dimensions [16, 22]. Three are decision factors: **Authorizations (A)** are the familiar attribute-based decisions on a subject's access to a resource; **Obligations (B)** are requirements that must be fulfilled by the subject for allowing access, such as accepting a licence agreement; and **Conditions (C)** are subject- and object-independent environmental requirements, such as time-of-day or system load, that must be satisfied for access [22].

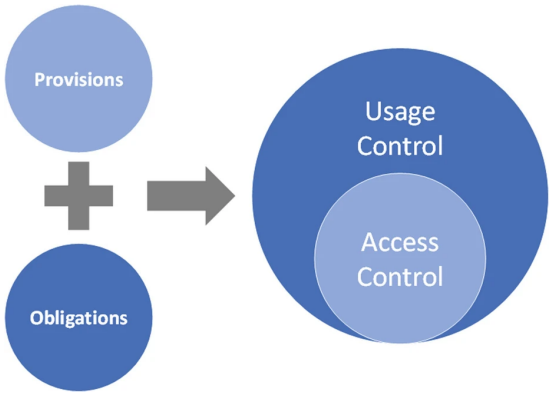

Figure 2.2: Usage control extends access control [16]

Two cross-cutting properties complement these decision factors. **Continuity** refers

to the ongoing evaluation and enforcement of authorisations throughout the usage of a resource, extending the traditional one-time, pre-access decision to continuous enforcement that can trigger revocation during active usage. **Mutability** refers to the ability for the attributes on which access decisions are based to be modified as a consequence of access, facilitating updates before (pre), during (ongoing), and after (post) the access decision [22]. Together, these five dimensions form the UCON family of ABC models, a core framework that subsumes traditional access control models, while also capturing newer requirements from trust management and digital rights management [22].

UCON+ and Continuous Authorisation

UCON+ is an improvement over the original UCON model that adds continuous monitoring before granting and after revoking authorisations, as well as policy administration and delegation [13]. As shown in fig. 2.3, the UCON+ architecture extends the XACML roles introduced in section 2.2 with additional components: a CH that receives and routes access requests and manages the authorisation workflow; a *Session Manager* (SM) that tracks active sessions and their lifecycle; an *Obligation Manager* (OM) that handles obligations that must be performed alongside enforcement decisions; and an *Attribute Table* (AT) that caches attribute values for periodic polling as part of continuous monitoring. The PDP, PAP, PIP, and PEP retain their XACML roles but operate within this extended workflow.

Central to UCON+ is the session lifecycle managed by the SM. Sessions transition through distinct phases: pre, ongoing, and post, depending on their authorisation workflow. When access is revoked, the session enters a *revoked* state, which persists until all obligations are fulfilled and access is terminated. This lifecycle enables continuous re-evaluation of policies during active usage, which is directly relevant to how DYNAMOS handles ongoing policy evaluation in this thesis.

2.5 BRANE: Programmable Orchestration with Policy Reasoning

The BRANE framework, developed by Valkering et al. [27] within the Multiscale Networked Systems (MNS) group at the University of Amsterdam, provides programmable orchestration of multi-site applications. BRANE uses containerisation to encapsulate functionalities as portable building blocks and offers a domain-specific language for expressing application orchestration.

Of particular relevance to this thesis is BRANE's integration of eFLINT-based policy reasoning. Esterhuyse et al. [8] explored enforcing private, dynamic policies on workflow execution within BRANE, demonstrating how normative specifications can govern data exchange and processing in distributed settings. While BRANE incorporates a policy reasoner based on eFLINT, it does not support dynamically changing the policies during runtime. DYNAMOS, on the other hand, lacks powerful reasoning capabilities but offers a suitable testbed for studying dynamically composed execution workflows driven by changing policies. This complementarity motivates the present thesis: integrating an eFLINT-based reasoner with DYNAMOS, drawing on knowledge from the BRANE project, to enable dynamic policy enforcement.

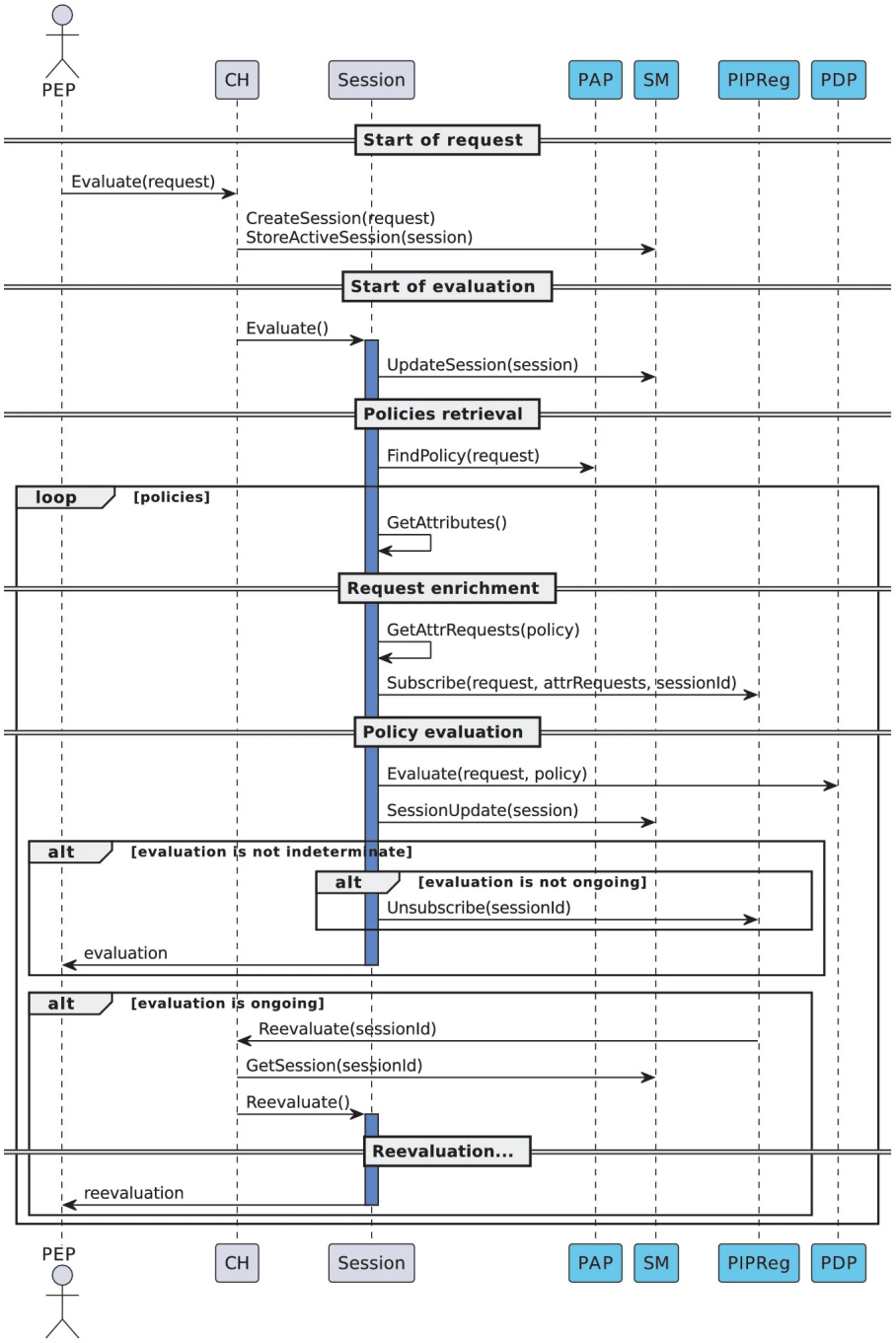

Figure 2.3: Sequence diagram of the core UCON+ workflow [13]

Chapter 3

Related Work

This chapter presents the most relevant research related to this thesis. It highlights key approaches in policy enforcement, policy languages, and usage control, and shows how this work compares to existing studies.

3.1 DDM Architectures and Enforcement

DYNAMOS (Stutterheim) Stutterheim [25] introduce DYNAMOS, a microservice-based DDM middleware. Its policy enforcer was a mocked implementation that validated requests by intersecting JSON fields stored in etcd, without normative reasoning, without re-evaluation during active sharing, and with blurred responsibilities between orchestrator and enforcer (chapter 4 and section 4.1). This thesis replaces that baseline with an eFLINT-based Policy Enforcer that applies normative derivation, re-evaluates in-flight sessions when a policy changes, and provides clearer separation of PDP, PAP, and PEP roles.

IDS and GAIA-X (Otto et al.) Otto et al. [18] describes data spaces as distributed ecosystems in which each participant retains sovereignty over its data and exchanges it peer-to-peer through connectors. Eitel et al. [7] specifies that IDS usage-control decisions are made inside each connector, using ODRL-based contracts that providers and consumers negotiate and deploy locally, in a self-governed arrangement in the sense of Binsbergen et al. [3], with no single platform-wide PDP. This thesis implements the contrasting intermediary-governed pattern from the same work: DYNAMOS centralises policy decisions in a single Policy Enforcer (eFLINT as PDP, etcd as PAP), so a policy can be updated at runtime from a single administration point and immediately reapplied to all active sessions. The evaluation confirmed this behaviour for centrally-managed eFLINT policies, an outcome the per-connector IDS model does not provide by design.

Semantic and infrastructure enforcement (Shakeri et al.) Shakeri et al. [23] model pre-established DDM agreements in an extended ODRL vocabulary and, in a matching module, use SPARQL queries to decide at request admission whether a sharing request is feasible and, if so, approve or reject it before infrastructure deployment. They do not

address ongoing re-evaluation of authorisation during an active sharing operation. In complementary work, Shakeri et al. [24] studies policy-compliant sharing in a container platform: matched policies are translated into network configuration, and per-request overlay networks improve isolation between concurrent requests; the contribution is infrastructure design and evaluation rather than policy reasoning itself. Neither work supports runtime policy mutation with mid-session re-evaluation. This thesis shares the request-admission goal but differs in reasoning mechanism (eFLINT action-based normative derivation rather than SPARQL matching over ODRL policies), in continuous authorisation (the Session Manager re-evaluates already-approved sessions when a policy changes, yielding no-op, recomposition, or revocation decisions), and in the enforcement layer (orchestrator middleware rather than primarily network overlays). The contrast in reasoning mechanism is therefore about what each system implements today: SPARQL-based matching over ODRL policies, as in Shakeri et al. [23], could in principle be encoded in eFLINT as well, but this thesis does not pursue that integration and instead relies on action-based normative derivation.

BRANE (Valkering et al.) BRANE [27] is the closest sibling system: a UvA DDM framework whose checker component likewise applies eFLINT for normative policy checking (section 4.3). As analysed there, BRANE separates a stable interface specification, an expert-authored *user policy* loaded from a database at each check, and per-request facts, but it does not provide the agreement-rule layer developed in this thesis: shared derivation rules and `Extend Fact` clauses that bridge the fixed query schema to consortium-specific steward agreements while translating DYNAMOS's existing JSON agreement logic into declarative normative derivations (chapter 6). The more consequential difference is governance topology: BRANE enforces in a decentralised, algorithm-travels-to-data arrangement aligned with the self-governed pattern of Binsbergen et al. [3], whereas this thesis realises the intermediary-governed counterpart (fig. 4.8), centralising decisions in a single Policy Enforcer. That centralisation supports runtime policy replacement with immediate re-evaluation of in-flight sessions, which BRANE does not support (section 2.5); it does, however, forego a decentralised, self-governed topology in which enforcement is distributed across sites and workflow phases rather than brokered through one platform-wide Policy Enforcer, a direction that warrants further investigation alongside central re-evaluation.

3.2 Policy Languages and Access Control

Access control landscape (Farhadighalati et al.) Farhadighalati et al. [11] surveys access control models and identifies six requirements (adaptability, context-awareness, distributed enforcement, dynamic access control, fine-grained control, flexibility), noting that no reviewed model covers all of them, and that normative languages such as eFLINT are not discussed. This thesis addresses four: runtime policy updates provide adaptability and dynamic access control; eFLINT derivations provide fine-grained per-request decisions with formal justification; the three-layer pattern provides flexibility. The two gaps are distributed multi-PDP enforcement (cf. the BRANE contrast above) and ongoing context-awareness: request facts supply contextual attributes at admission,

but the Session Manager re-evaluates in-flight sessions only on policy replacement, not when situational attributes change elsewhere (F3). The `permitted-continuation` path built for that re-evaluation could extend to attribute-change triggers once DYNAMOS exposes monitoring hooks, without further enforcer changes (section 6.2); until then, continuity in the usage-control sense remains event-triggered on policy updates only (section 3.3).

Limitations of ODRL and the case for eFLINT (Kebede et al.) Kebede et al. [17] argue from practical use cases that ODRL lacks expressiveness for dynamic normative change: it cannot represent action-triggered updates such as consent withdrawal, the revocation of delegated rights along open-ended chains, or the institutional effects of actions that grant or remove permissions. They propose eFLINT as a candidate language because its action-based, Hohfeld-grounded primitives better support legal power and normative state change over time. This thesis does not replicate their language-level comparison, but it addresses one operational consequence of that gap in DYNAMOS: when a steward replaces the agreement-layer policy at runtime, the Policy Enforcer and Session Manager re-evaluate active sessions against the new specification and classify each affected job as unchanged, requiring recomposition, or no longer authorised, which aligns with the mid-session withdrawal of permission they discuss (permission granted at request time may cease to hold during ongoing use) but is realised through event-triggered policy updates and continuous authorisation (section 3.3), not through explicit modelling of withdrawal or delegation revocation as institutional actions in the agreement rules. Delegation-chain revocation and general action-triggered policy mutation, therefore, remain outside the scope of this work.

eFLINT-based enforcement (van Binsbergen et al.) van Binsbergen et al. [28] formalise the GDPR and a healthcare consortium's regulatory document in eFLINT and show how compliance decisions can be automated; they explicitly flag DDM integration with mid-session consent withdrawal as future work. Binsbergen et al. [3] then propose a theoretical XACML/eFLINT architecture with delegation archetypes, self-governed and intermediary-governed topologies, and three auditability requirements (process only when lawful; record derivation trees; log all processing). Neither paper implements this in a running DDM middleware. This thesis realises both: it fulfils the DDM future work of van Binsbergen et al. [28] and concretely implements the XACML/eFLINT architecture of Binsbergen et al. [3] within DYNAMOS, with eFLINT as PDP, etcd as PAP, and sidecars as PEPs. The evaluation confirmed all three auditability requirements are satisfied; in particular, every `ValidationResponse` carries a structurally correct derivation tree.

3.3 Usage Control and Continuous Authorisation

Sandhu et al. [22] introduces *continuity* (ongoing re-evaluation throughout usage) and *mutability* (policy attributes can change as a result of actions) as the defining extensions of usage control over access control. This thesis implements both in an event-triggered form: continuity via the Session Manager, which re-evaluates active sessions

when a policy update arrives, and mutability via the runtime policy update mechanism. The request-approval orientation of the evaluation confirms that policy decisions are correctly enforced (section 7.2); evaluation of the continuous-authorisation and dynamic-update orientations is left as future work. Only event-triggered continuity was implemented, not the stronger time-interval-based form.

Hariri et al. [13] concretises continuity in UCON+: a modular, language-agnostic system with session lifecycle management (ucon-c), ALFA-based policy evaluation (libalfa), and a gRPC service interface. This thesis shares UCON+'s architecture of separating session management from policy reasoning, but substitutes eFLINT for ALFA, providing normative derivation trees and legal-logic grounding that ALFA does not offer, and uses event-triggered rather than attribute-polling re-evaluation. Reina Quintero et al. [20] also proposes a DSL for UCON policies, but with a fixed vocabulary of six rule types (AllowRule, StopRule, and four obligation/condition variants) enforced via model transformations at design time. Unlike their closed vocabulary, eFLINT lets any consortium agreement be expressed as arbitrary fact-, duty-, and action-types; and unlike their offline model-transformation approach, this thesis evaluates eFLINT at runtime, enabling a data steward to replace the agreement-layer policy without touching the codebase. Eitel et al. [7] take yet a different approach in the IDS: ODRL-based usage contracts enforced at each connector, with policy changes requiring contract re-negotiation. This thesis centralises enforcement and supports immediate runtime updates without re-negotiation, at the cost of the decentralised autonomy that both UCON+ and the IDS usage control framework provide.

Chapter 4

Analysis

This chapter establishes the analytical foundation for informed design decisions in chapter 5. Section 4.1 examines the DYNAMOS middleware platform, covering its architectural components, the two core workflows (request approval and changed agreement), and the existing policy evaluation logic. Within the same section, section 4.1 maps the DYNAMOS components onto the XACML data-flow architecture to provide a standardised lens for evaluating the current design; the structural gaps that emerge are consolidated as concrete shortcomings in section 4.1. Next, section 4.2 analyses the eFLINT policy reasoner by walking through its core language constructs, establishing the vocabulary needed to understand the design in chapter 5. Finally, section 4.3 examines the BRANE framework and draws lessons from its integration of the eFLINT reasoner.

4.1 DYNAMOS Middleware

DYNAMOS, *Dynamically Adaptive Microservice-based OS*, is a middleware platform for distributed data exchange, introduced by Stutterheim [25]. "Inspired by the way traditional Operating Systems abstract the complexities of computer architectures into standardized core functions" [25], DYNAMOS abstracts common data-exchange patterns, called *archetypes*, into a unified set of composable data-exchange microservices that operate under agreed-upon policy. The platform is designed to be self-adaptive: it can dynamically select an archetype and microservice composition per request, adjusting to evolving agreements, user input, or changes in system load.

DYNAMOS runs on Kubernetes, deploying all components as containerised microservices. External clients communicate with the platform over HTTP through an *API Gateway*; internally, the API Gateway, orchestrator, policy enforcer, and distributed agents communicate using gRPC. The infrastructure layer relies on several supporting tools: *etcd* serves as a distributed key-value store for static configuration and reactive state (e.g. active composition requests and agreements); *RabbitMQ* provides an asynchronous messaging backbone; *Linkerd* handles service-mesh concerns and traffic observability; and *Jaeger*, *Prometheus*, and *Grafana* provide distributed tracing, metrics collection, and dashboarding, respectively.

At the application level, DYNAMOS is composed of two primary logical tiers, as

depicted in fig. 4.1. The *orchestration tier* contains the *Orchestrator* and the *Policy Enforcer*: the orchestrator drives request routing and dynamic microservice-chain composition, while the policy enforcer validates requests against the data-sharing agreements stored in etcd. Stutterheim [25] notes that the current policy enforcer is a mock implementation of the request validation [25]. The *data-steward tier* can contain one or more data stewards, each hosting a *Distributed Agent* that coordinates local execution, alongside a set of dynamically composed microservice chains that perform the actual data processing. Within each data steward, microservices communicate via a sidecar.

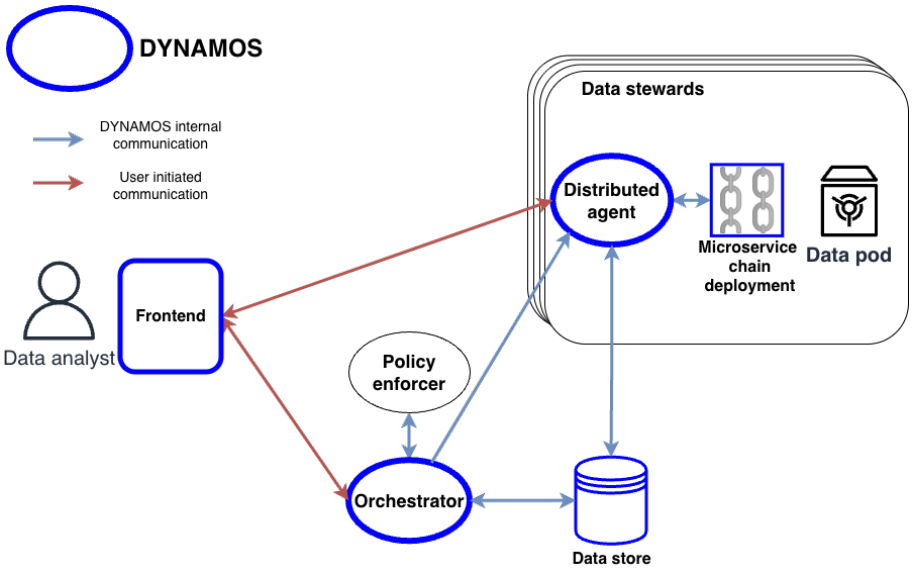

Figure 4.1: Abstract architecture of the DYNAMOS middleware platform by Stutterheim [25]

DYNAMOS targets interoperability within the AMdEX ecosystem [25]. The AMdEX Reference Architecture [30], depicted in fig. 4.2, divides a dataspace into three planes: the *data plane* (bottom), the *control plane* (top), and the *governance plane* (top-right).

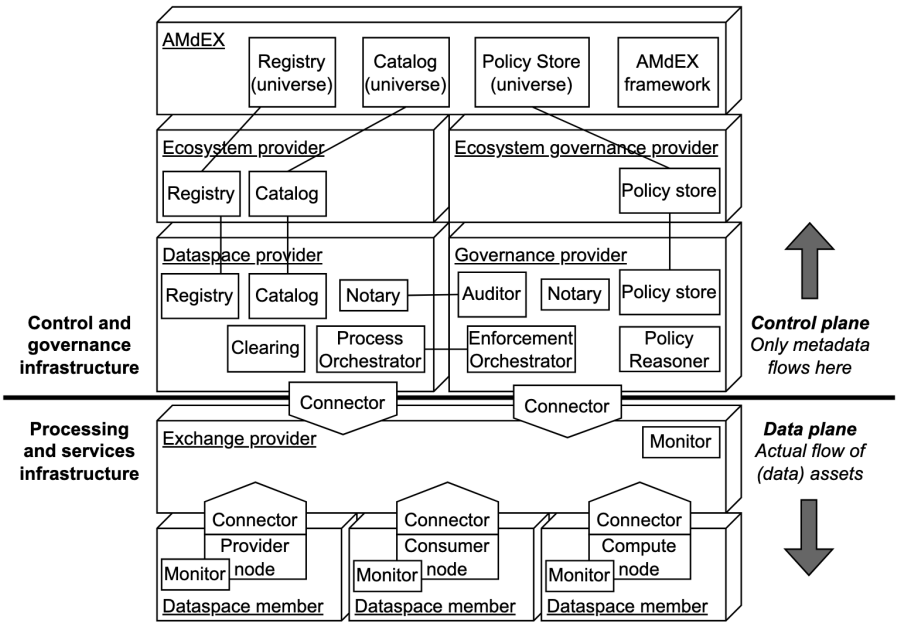

Figure 4.2: AMdEX Reference Architecture: data plane, control plane, and governance plane [30].

Stutterheim et al. [26] maps DYNAMOS explicitly onto this three-plane division, as shown in fig. 4.3. RabbitMQ serves as the exchange provider in the data plane, while the distributed agents and their data stewards operate as dataspace members. The orchestrator is situated in the control plane, and the policy enforcer occupies the governance plane. This placement will be further refined in section 4.1, where XACML usage control components are mapped onto the same architecture.

Data Request Approval Workflow

Figure 4.4 shows how a data request moves through DYNAMOS. The data analyst calls the API Gateway over HTTP (`POST /api/v1/requestApproval`); traffic between the analyst and the gateway is HTTP, while the API Gateway, orchestrator, policy enforcer, and distributed agents communicate internally using gRPC. The gateway forwards a *RequestApproval* to the orchestrator, which validates the request against policy by invoking the policy enforcer (*PolicyValidation* and *ValidationResponse*). After approval, the orchestrator issues a *CompositionRequest* to the relevant distributed agent(s) and returns an *RequestApprovalResponse* to the API Gateway. For each distributed agent, the API Gateway then sends a *SqlDataRequest*; steps corresponding to *RequestApprovalResponse*/*CompositionRequest* and *SqlDataRequest* thus repeat as many times as there are distinct data stewards involved in the analyst's request. Within each data steward, the distributed agent drives execution via the sidecar (*MicroserviceCommunication*). The

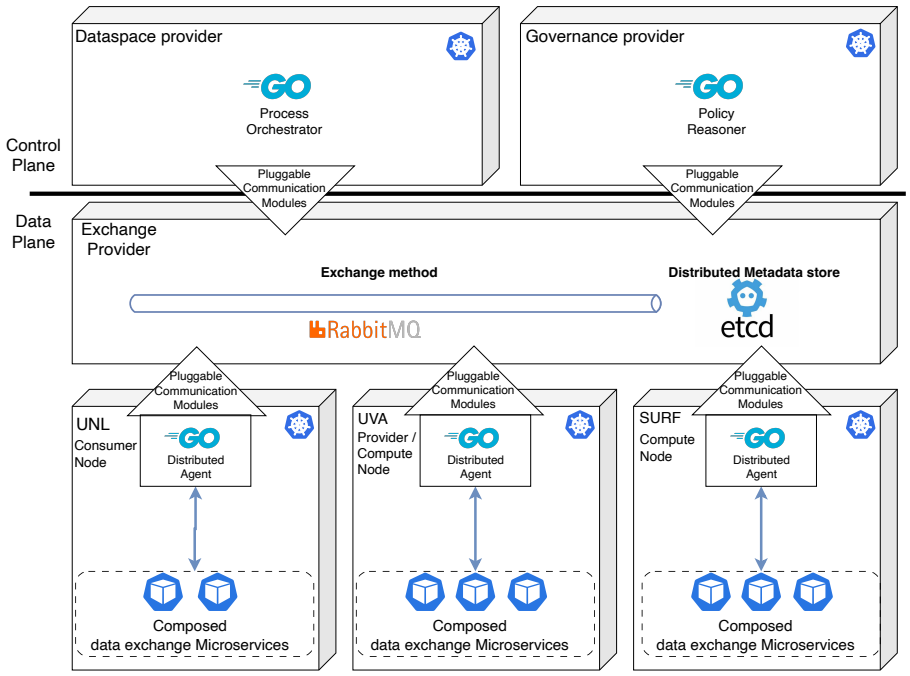

Figure 4.3: DYNAMOS mapped onto the AMdEX three-plane architecture [26].

distributed agent returns a *Response* to the API Gateway, which consolidates multiple responses when several stewards participated, and sends the final result to the data analyst as an HTTP response. On top of this diagram we can also project the data analyst request flow into a sequence diagram in Figure 4.5.

Changed Agreement Workflow

On top of the data request approval workflow, the current DYNAMOS architecture provides a second policy-relevant workflow: the *changed agreement workflow*. This workflow is triggered whenever an existing agreement is updated, via an HTTP `PUT /api/v1/policyEnforcer/agreements` call, and must reconcile the updated agreement with any data-exchange jobs that are already in flight.

The workflow is illustrated in fig. 4.6. When an *AgreementChanged* event is raised (step 1), the orchestrator first queries etcd for all active jobs (*CheckActiveJobs*, step 2). These active jobs are tracked in etcd by the distributed agents, which store the composition requests they received during the original request approval. The orchestrator then forwards a *PolicyUpdate* message to the policy enforcer (step 3), which evaluates the updated agreement and returns a *PolicyUpdate* response that embeds a `validation_response` (step 4). The orchestrator uses this response to decide, for each active job related to the changed agreement, whether the job remains valid under the new agreement. Finally, the orchestrator writes the updated job state back to etcd (*UpdateActiveJobs*, step 5). Jobs that are no longer valid are either recomposed or cancelled.

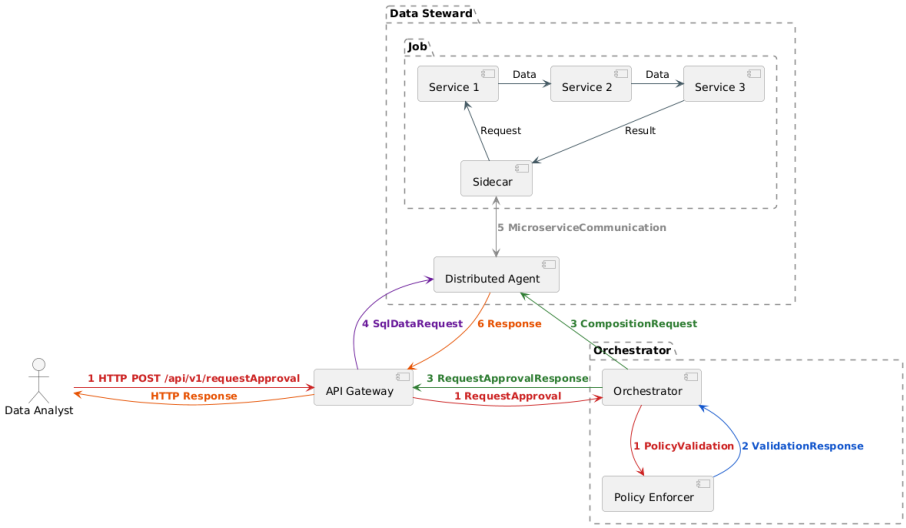

Figure 4.4: Data analyst request flow of the DYNAMOS middleware platform

Compared with the request approval workflow, the changed agreement workflow shares the same policy-evaluation mechanism at its core: in both cases, the policy enforcer produces a *ValidationResponse* that the orchestrator acts on. The structural difference is in the envelope: during request approval the policy enforcer replies with a standalone *ValidationResponse* protobuf, whereas here the same validation payload is carried inside a *PolicyUpdate* message.

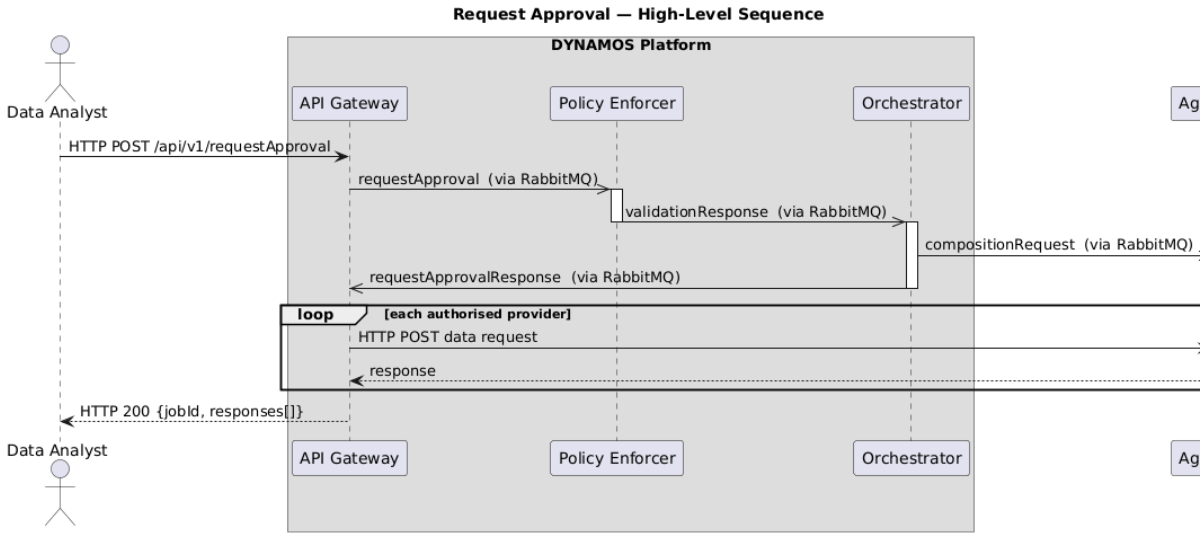

Figure 4.5: Sequence diagram of the data analyst request flow of the DYNAMOS middleware platform

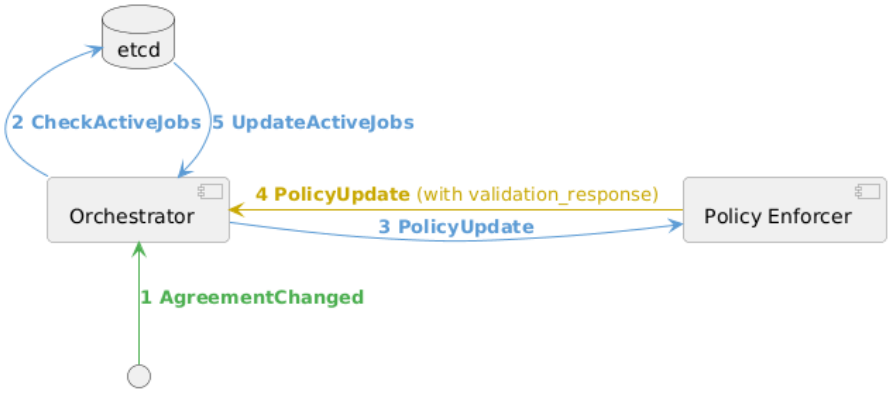

Figure 4.6: Changed agreement workflow of the DYNAMOS middleware platform

Policy Evaluation

As noted in section 4.1, Stutterheim explicitly identifies the policy enforcer as a mock implementation [25]. Rather than applying formal normative reasoning, it evaluates each incoming request by directly inspecting the JSON-structured agreements stored in etcd. The following describes the messages the policy enforcer consumes, the agreement structure it reads, and the internal logic it applies to produce its output.

Both message flows described above converge on the same internal evaluation logic inside the policy enforcer; only the envelope differs. The following listings detail the exact protobuf structures involved. Listing 1 shows the `RequestApproval` protobuf

message that the API Gateway sends to the policy enforcer. It specifies the requesting user and the list of data providers whose agreements must be checked.

```
// RequestApproval (API Gateway -> Policy Enforcer)
message RequestApproval {
  string type = 1;                    // "requestApproval"
  User user = 2;                      // id, user_name
  repeated string data_providers = 3; // e.g. ["VU", "UVA"]
  string destination_queue = 4;       // "policyEnforcer-in"
  map<string, bool> options = 5;
}
```

Listing 1: `RequestApproval` protobuf message (API Gateway → Policy Enforcer).

Listing 2 shows the `ValidationResponse` the policy enforcer returns to the orchestrator. For each requested data provider it records which archetypes and compute providers are permitted (`valid_dataproviders`), which providers were rejected (`invalid_dataproviders`), and whether the overall request is approved. When at least one data provider passes validation, `request_approved` is set to `true` and a short-lived authentication token pair is generated.

```
// ValidationResponse (Policy Enforcer -> Orchestrator)
message ValidationResponse {
  string type = 1;                                  // "validationResponse"
  string request_type = 2;                          // e.g. "sqlDataRequest"
  map<string, DataProvider> valid_dataproviders = 3; // per steward: archetypes[],
  repeated string invalid_dataproviders = 4;        //    compute_providers[]
  Auth auth = 5;                                    // access_token, refresh_token
  User user = 6;                                    // id, user_name
  bool request_approved = 7;
  UserArchetypes valid_archetypes = 8;              // user_name, archetypes per steward
  map<string, bool> options = 9;
}
```

Listing 2: `ValidationResponse` protobuf message (Policy Enforcer → Orchestrator).

As described above, the changed agreement workflow reuses the same evaluation logic but wraps the exchange in a `PolicyUpdate` message (listing 3). When the orchestrator sends a `PolicyUpdate` to the policy enforcer, the `validation_response` field is empty; when the policy enforcer returns it, `validation_response` is populated with the full evaluation result.

The internal evaluation logic applied to both message types is depicted in fig. 4.7. Upon receiving an incoming message from the `policyEnforcer-in` queue, the policy enforcer initialises an empty `ValidationResponse` and iterates over each requested data provider. For each provider, it queries etcd at `/policyEnforcer/agreements/{steward}` and retrieves the corresponding agreement. If the agreement exists and the requesting user is listed in its `relations`, the enforcer intersects the archetypes permitted

```
// PolicyUpdate (Orchestrator <-> Policy Enforcer)
// validation_response is empty outbound, populated inbound.
message PolicyUpdate {
  string type = 1;
  User user = 2;
  repeated string data_providers = 3;
  RequestMetadata request_metadata = 4;
  ValidationResponse validation_response = 5;
}
```

Listing 3: `PolicyUpdate` protobuf message used in the changed agreement workflow (Orchestrator ↔ Policy Enforcer).

by the user's relation entry with those declared by the data steward at the top level of the agreement. Only archetypes present in both sets are considered valid; the corresponding compute providers are stored alongside them. If no agreement exists, the user is absent from the relations, or no archetypes match, the data provider is added to `invalid_dataproviders`. After iterating over all requested providers, `request_approved` is set to `true` if at least one valid provider was found, and the completed response is published to the `orchestrator-in` queue.

Listing 4 shows a concrete agreement as stored in etcd. The top-level `archetypes` and `computeProviders` fields declare what the data steward supports in general. The `relations` map then constrains individual users: each entry specifies which request types, datasets, archetypes, and compute providers that particular user is allowed. The policy enforcer intersects the user's `allowedArchetypes` with the steward's `archetypes` to arrive at the set of valid archetypes for the request.

```
// etcd key: /policyEnforcer/agreements/UVA
{
    "name": "UVA",
    "relations": {
        "n.p.arts@uva.nl": {
            "ID": "GUID",
            "requestTypes": ["sqlDataRequest", "genericRequest"],
            "dataSets": ["wageGap"],
            "allowedArchetypes": ["computeToData", "dataThroughTtp"],
            "allowedComputeProviders": ["SURF"]
        }
    },
    "computeProviders": ["SURF", "otherCompany"],
    "archetypes": ["computeToData", "dataThroughTtp", "reproducableScience"]
}
```

Listing 4: Example agreement for a data steward stored in etcd

The agreement structure and the intersection logic illustrated above show *how* de-

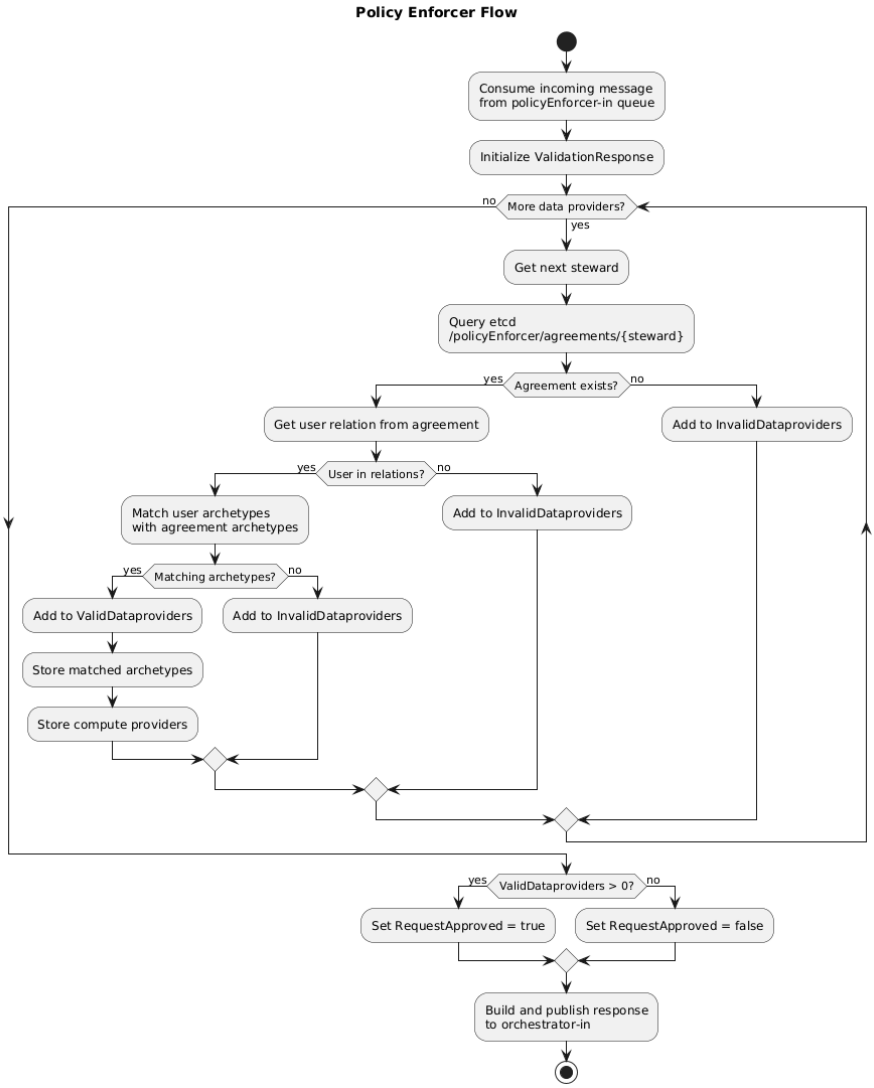

Figure 4.7: Policy enforcer existing workflow of the DYNAMOS middleware platform

cisions about datasets, archetypes, and compute providers can be made, and the resulting `ValidationResponse` provides the orchestrator with the information it needs to compose the microservice chain. Decisions are determined purely by the static JSON structure stored in etcd, without any formal normative reasoning. This agreement structure and its validation logic will serve as the reference point when extending the policy enforcer with a normative policy specification in chapter 5.

Two structural issues emerge from the changed agreement workflow. First, because the orchestrator selects jobs from etcd based on composition requests registered by

distributed agents, jobs that have not yet been registered or whose entries have expired may be silently missed. Second, and more fundamentally, determining which running jobs are affected by a policy change is a responsibility that belongs to the policy enforcer, not the orchestrator. This lack of separation of concerns will be addressed in the design of the extended policy enforcer in chapter 5.

Mapping to the XACML Data-Flow Architecture

The XACML data-flow model defines five logical components: the Policy Enforcement Point (PEP), Policy Decision Point (PDP), Policy Administration Point (PAP), Policy Information Point (PIP), and Context Handler (CH). Although DYNAMOS was not designed with XACML in mind, its components align closely with this model.

The **API Gateway and sidecar proxies** fulfil the PEP role. They intercept incoming data requests and enforce the outcome of the authorization decision, without participating in the decision itself. The sidecar pattern, explicitly adopted in DYNAMOS, is a natural fit for this responsibility as it decouples enforcement from business logic.

The **Policy Enforcer** fulfils the PDP role. It receives the structured request context from the Orchestrator, evaluates it against the applicable policy agreement, and produces an access decision.

The **Orchestrator** consequently maps to the CH. It receives the raw incoming request, resolves its type, assembles the necessary context, and coordinates the flow, without making the authorization decision itself.

The **etcd-backed configuration and agreement store** serves as the PAP. Policies and agreements are authored, stored, and distributed to the Policy Enforcer through this layer. The Evolving Agreement mechanism [25] represents the PAP actively propagating policy changes at runtime.

Finally, the **Distributed Agents**, together with the shared etcd state, act as the PIP. They expose the runtime attributes (node availability, system load, participant roles) that the Policy Enforcer requires to make a well-informed authorization decision.

XACML component	DYNAMOS component
PEP	API Gateway + Sidecar proxies
PDP	Policy Enforcer
PAP	etcd + agreement/configuration store
PIP	Distributed Agents + etcd runtime state
CH	Orchestrator

Table 4.1: Mapping of XACML data-flow components to DYNAMOS.

As shown in Table 4.1, DYNAMOS covers all five XACML roles. DYNAMOS does not reference XACML directly, but targets AMdEX interoperability [25], which itself builds on XACML [30], making the alignment inherited rather than coincidental.

Building on the AMdEX architecture introduced in fig. 4.2 and the DYNAMOS placement shown in fig. 4.3, fig. 4.8 from Binsbergen et al. [3] shows how XACML usage control components can be mapped onto the same AMdEX planes. This figure combines

the distributed-processing archetype with the control-plane / data-plane division: the Policy Enforcer (intermediary node) and the controller hosting the PEP both reside in the control plane, while data flows in the data plane below. The progression across these three figures, AMdEX reference architecture, DYNAMOS on AMdEX, and the XACML component mapping, establishes the architectural foundation on which the design in chapter 5 builds.

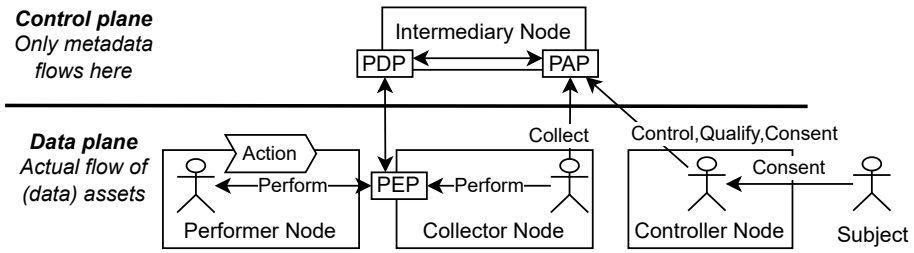

Figure 4.8: Intermediary Node: Distributed-processing archetype with control-plane / data-plane division [3].

Shortcomings of the current policy enforcer

Three shortcomings follow directly from the analysis above.

No normative reasoning. The policy enforcer is a mock implementation [25]. Rather than reasoning about permissions, obligations, or prohibitions in any formal sense, it resolves requests by intersecting JSON-encoded archetype lists stored in etcd. There is no concept of legal basis, purpose limitation, or consent, and no obligations can be attached to an access grant. The current evaluator handles *authorisation* in the technical sense but not *permission* in the legal sense [3].

Pre-access enforcement only. Beyond the changed agreement workflow, validation occurs only once, before the data request is approved. Once a `ValidationResponse` is returned with `request_approved = true`, no further monitoring or control is exercised over the ongoing data exchange. The continuity and mutability properties of the UCON model are therefore unsupported: changes in environmental conditions, user attributes, or resource attributes that affect the validity of a decision are not re-evaluated while a job is running.

Separation-of-concerns violation. As identified in section 4.1, the orchestrator (CH) currently determines which active jobs are affected by a policy change, a responsibility that belongs to the policy enforcer (PDP). Beyond violating the XACML role boundaries established in section 4.1, this design risks silently missing jobs whose etcd entries have not yet been registered or have already expired.

These three shortcomings; the absence of normative reasoning, the lack of ongoing or continuous enforcement, and the misplaced policy logic, motivate the design of an extended policy enforcer in chapter 5.

4.2 eFLINT Policy Reasoner

Section 2.3 established that eFLINT is an action-oriented DSL grounded in Hohfeld's framework, and that an eFLINT program consists of a *normative specification* and a *scenario description* [3, 29]. This section unpacks the concrete mechanics of those two parts using three examples from Binsbergen et al. [3], establishing the vocabulary needed to understand the design in chapter 5.

Listing 5 shows a normative specification that defines a small family-relationship ontology with derivation rules. `Fact` declares a type; `Identified by A * B` makes it relational. `Holds when` attaches a derivation rule; `sibling` is inferred automatically, never asserted directly. `Extend Fact` adds rules to an existing type without redefining it.

```
Fact person  Identified by Alice, Bob, Chloe  // type has three instances (atoms)
Fact sister  Identified by person1 * person2  // binary relation over persons
Fact brother Identified by person1 * person2
Fact sibling Identified by person1 * person2
  Holds when sister(person1, person2),        // rule SISTER
             brother(person1, person2)        // rule BROTHER
Extend Fact brother
  Holds when sister(person2, person1)         // rule SYMMETRY
```

Listing 5: eFLINT normative specification: ontology with derivation rules (Listing 1 in Binsbergen et al. [3]).

Listing 6 introduces action- and duty-types. An `Act`'s `Holds when` is its *enablement condition*; `Creates` is its *normative effect*, here asserting a `Duty` into the normative state. This embodies Hohfeld's power-duty relation introduced earlier in section 2.3: the act exercises a power that places an obligation on another party.

```
Act ask-for-help
  Actor     person1
  Recipient person2
  Holds when sibling(person1, person2)       // enabled only between siblings
  Creates duty-to-help(person2, person1)     // effect: creates obligation

Duty duty-to-help
  Holder   person1  // bears the obligation
  Claimant person2  // entitled to demand performance
```

Listing 6: eFLINT action-type and duty-type declaration (Listing 2 in Binsbergen et al. [3]).

Listing 7 shows the scenario operating on the specification above. `+fact()` creates a fact; a bare `act()` fires an action. `?Holds()` is a boolean query mapping to a PDP

permit/deny decision. `?-var When condition` is a generative query that enumerates all satisfying instances, enabling the reasoner to return a set of permitted values rather than a single decision.

```
+sister(Chloe , Bob).              // event trigger for adding a fact to the KB
                                   // various facts are derived,
                                   // including sibling(Bob, Chloe), ask-for-help(Bob, Chloe)
?Holds(ask-for-help(Bob, Chloe)).  // Boolean query succeeds
?Holds(ask-for-help(Bob, Alice)).  // Boolean query fails
ask-for-help(Bob, Chloe).          // action triggered by the event trigger
                                   // the duty to help Bob is created for Chloe
?Holds(duty-to-help(Chloe, Bob)).  // Boolean query succeeds
?-person When sibling(Bob, person). // query generating all siblings of Bob
                                   // yields: person(Chloe)
```

Listing 7: eFLINT scenario: triggers and queries (Listing 3 in Binsbergen et al. [3]).

4.3 BRANE Framework

Section 2.5 introduced the BRANE framework as a programmable orchestration platform and noted that Esterhuyse et al. [8] integrated an eFLINT-based policy reasoner into it. This section examines the concrete implementation of that integration inside `brane-chk`, the dedicated checker component, to extract design lessons applicable to DYNAMOS.[1]

`brane-chk` separates policy concerns into three layers. The *interface/base policy* is a stable eFLINT specification that defines the ontology and the fixed set of facts that can be queried; it changes only when the system's domain model changes. The *user policy* is the runtime normative specification authored by a policy expert and stored in a database; it can be updated freely without touching the system. The *request facts* are ephemeral eFLINT facts generated per call from the incoming workflow context (nodes, datasets, users, recipients). This separation embodies the principle articulated by Binsbergen et al. [3]: "By separating the rules formalising the norms from the implementation of a system, the transparency, adaptability and maintainability of the system are increased."

`brane-chk` exposes three HTTP endpoints, each intercepting a distinct point in the workflow: `/v2/workflow`, `/v2/task`, and `/v2/transfer`. Incoming requests pass through JWT validation before reaching the resolver. The resolver loads the active user policy from the database, verifies that its declared language matches the current reasoner base hash (`eflint-haskell-{hash}`), compiles the workflow intermediate representation (WIR) into a reasoner workflow model, and constructs one of four *questions*:

- **ValidateWorkflow** (`/v2/workflow`): triggered during planning, intended as a global pre-execution workflow check.
- **ExecuteTask** (`/v2/task`): triggered immediately before a specific task is dispatched to a worker.

[1] Source code can be found at https://github.com/BraneFramework/brane/tree/main [4]

- **TransferInputs** (`/v2/transfer`, task present): triggered when a registry authorises a task's input data download.
- **TransferResult** (`/v2/transfer`, task absent): triggered when a registry authorises the final result download to the workflow user.

Each question is rendered as an eFLINT scenario: a block of request facts (e.g. `+workflow(...)`, `+node(...)`) followed by a single boolean query against one of the four facts defined in the interface policy, shown in listing 8.

```
Fact workflow-to-execute  Identified by workflow
Fact task-to-execute      Identified by task
Fact dataset-to-transfer  Identified by node-input
Fact result-to-transfer   Identified by workflow-result-recipient
```

Listing 8: Interface-policy query facts in `brane-chk`. Each check endpoint queries exactly one of these facts.

The reasoner connector then calls the eFLINT REPL (the Haskell implementation) with `state` set to the active user policy text and `question` set to the generated request facts plus the final `?Holds(...)` query. The result maps directly to a permit or deny decision.

The key property of this design is that the query schema is the only fixed interface between the system and the policy expert. Because the interface policy pins the fact names, a policy expert can rewrite the entire user policy without any code change, and the checker will continue to function correctly. This approach will directly inform the design of the extended DYNAMOS policy enforcer in chapter 5.

Chapter 5

Designing the Policy Enforcer

This chapter answers SRQ1 and SRQ2 by translating the analysis of chapter 4 into a concrete architecture. Section 5.1 derives the requirements that the new policy enforcer must satisfy. Section 5.2 presents the architecture that addresses SRQ1: the internal decomposition of the policy enforcer, its placement within DYNAMOS, and the interface to the Policy Reasoner. Section 5.3 addresses SRQ2 by describing how policy changes propagate at runtime and how they alter the execution of in-flight data-sharing operations.

5.1 Architectural Requirements

The requirements below are derived primarily from the shortcomings identified in chapter 4 (notably the lack of normative reasoning, the absence of continuous enforcement, and the separation-of-concerns violation between the orchestrator and the enforcer), as well as from the practical constraints of integrating a new enforcer into DYNAMOS. The literature in chapter 3 and chapter 2 is used to validate and sharpen specific requirements—in particular, role separation (XACML/UCON alignment) and auditability properties for lawful, accountable decisions [3]. Each requirement is labelled F (functional) or N (non-functional) and is referenced from the architectural choices that follow.

F1 – Normative reasoning. The enforcer must evaluate requests by reasoning over a normative specification expressed in eFLINT, supporting permissions, prohibitions, obligations and duties, rather than by a structural intersection of JSON fields.

F2 – Runtime-mutable policies. The active normative specification (the *agreement policy*) must be replaceable at runtime without restarting any DYNAMOS component and without recompiling the enforcer.

F3 – Continuous authorisation. Decisions must be re-evaluated for in-flight jobs whenever the policy they depend on changes, or when the attributes of the requester or the requested stewards change, supporting the continuity and mutability properties of the UCON model.

F4 – **Decision routing for in-flight jobs.** The enforcer must determine which active sessions are affected by a policy change and produce a per-session decision; this responsibility must not be delegated to the Context Handler.

F5 – **Obligation and duty tracking.** The enforcer must keep track of obligations and duties generated by the normative policy, recording their assignments and state transitions, even if active obligation enforcement is deferred to a future extension (e.g., an Obligation Manager).

F6 – **Stable query interface.** The set of facts that the runtime queries must form a fixed interface, independent of the agreement policy, so that policy authors can rewrite the agreement policy without changes to the enforcer or the Context Handler.

F7 – **Auditable decisions.** Every decision must be accompanied by a structured justification (the derivation tree produced by the reasoner) and persisted, satisfying three auditability conditions: lawfulness must be established before processing, the manner of establishment must be recorded, and processing must be logged [3].

N1 – **XACML role conformance.** The decomposition must respect XACML role boundaries: PEP, PDP, PAP, PIP, and CH must be distinguishable, and policy logic must reside in the PDP (chapter 4).

N2 – **Backwards compatibility.** Existing public interfaces of DYNAMOS (the API Gateway HTTP routes and the orchestrator/distributed-agent gRPC contracts) must remain functional. Internal protobuf messages may be extended additively.

N3 – **Bounded performance overhead.** The cost of policy evaluation on the request-approval path must remain within a bounded overhead budget that preserves platform responsiveness; concrete targets are defined and measured (chapter 7).

N4 – **Graceful degradation.** A reasoner failure must not silently admit requests; the enforcer must deny by default and surface the failure to the Context Handler.

5.2 Policy Enforcer Architecture (RQ1)

Component Overview

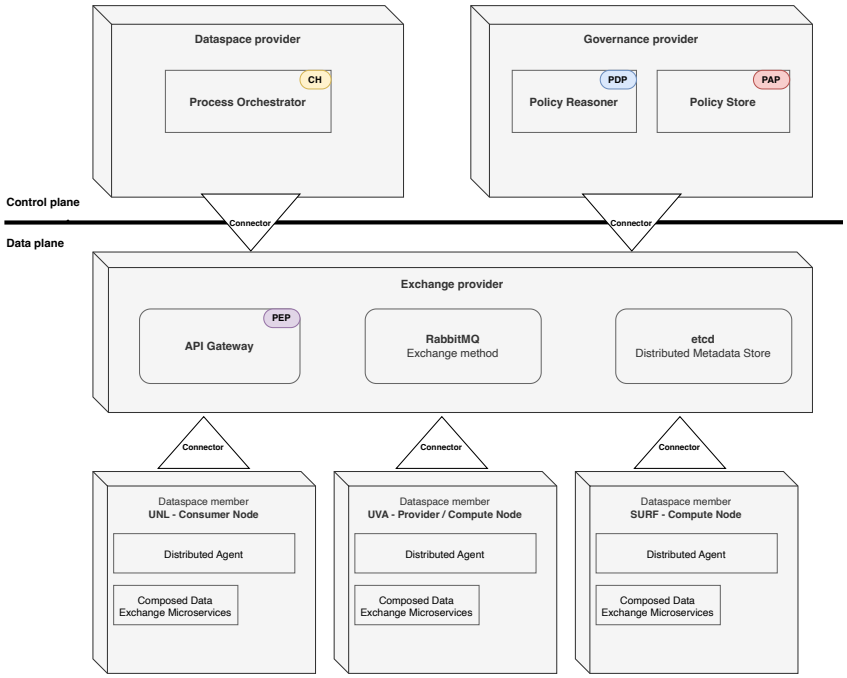

Figure 5.1: DYNAMOS mapped onto the AMdEX three-plane architecture with XACML usage-control components.

Figure 5.1 situates the new Policy Enforcer within the full DYNAMOS deployment by overlaying the XACML role assignment established in section 4.1 (summarised in table 4.1) onto the AMdEX three-plane architecture. The *governance plane* (top-right) is occupied by the Governance provider, which hosts the Policy Enforcer as PDP and the Policy Store as PAP. Policy decisions are therefore co-located with policy storage: the PAP makes the active agreement available to the PDP. The *control plane* (top-left) holds the Dataspace provider, whose Process Orchestrator acts as CH: it receives the raw request, assembles contextual attributes, and forwards a structured authorisation request to the PDP, without participating in the policy evaluation itself. The *data plane* (bottom) contains the Exchange provider and the three Dataspace members (UNL consumer node, UVA provider/compute node, and SURF compute node). The API Gateway in the Exchange provider and the fulfil the PEP role: it intercepts data requests at the plane boundary and enforce the PDP's decision without containing any policy logic. This plane-aligned placement satisfies requirement N1 structurally: each XACML role is confined to a single architectural layer, and the policy logic remains entirely within the Governance

provider.

Figure 5.2 shows the proposed architecture. The existing mock policy enforcer is replaced by a new *Policy Enforcer* composed of six internal components and an internal *Policy Reasoner* sub-component that fulfils the PDP role. The Policy Reasoner runs as a binary process inside the Policy Enforcer; to avoid the latency of cold-starting the binary on every incoming request, it maintains an *instance pool* – a set of pre-started reasoner instances that are ready to handle evaluations immediately. The decomposition follows directly from the requirements: each component either fulfils an XACML or UCON+ role (N1) or implements a capability (F1–F7) absent from the current enforcer.

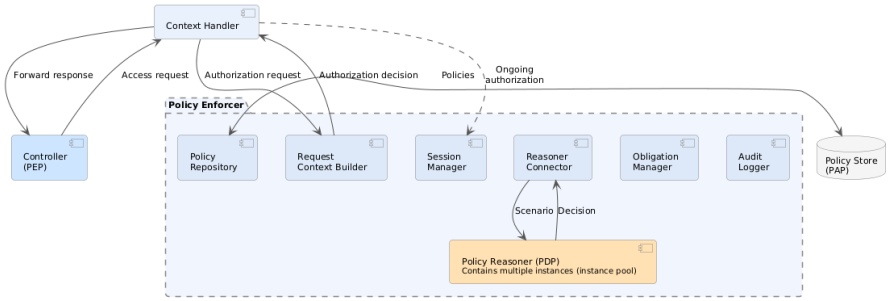

Figure 5.2: Policy Enforcer internal decomposition and immediate context.

The six internal components are summarised below; each entry names the requirements it primarily addresses and, where applicable, the corresponding XACML [10] or UCON+ [13] role.

Request Context Builder. Acts as the local entry point of the Policy Enforcer. It consumes incoming request and policy-update messages, extracts the relevant subject, object and environmental attributes, and assembles them into the structured *request facts* expected by the interface policy. Forward-looking attributes such as *purpose* [3] can be added without changes to the architecture elsewhere, provided the interface policy is extended accordingly. The Builder realises the local Context Handler responsibility identified by Hariri et al. [13] for inputs originating inside the enforcer; cross-service context routing remains the responsibility of the external Context Handler (N1).

Policy Repository. Holds the active normative specification in two layers, mirroring the separation introduced by Esterhuyse et al. [8] in brane-chk and discussed in chapter 4: a stable *interface policy* that fixes the names and arities of the facts the runtime queries (F6), and a runtime-mutable *agreement policy* that encodes the actual normative specification (F2). The Repository persists both layers in the *Policy Store* (PAP) and versions each agreement policy so that decisions can always be traced back to the specification that produced them.

Reasoner Connector. Mediates between the rest of the Policy Enforcer and the internal Policy Reasoner. It loads the active specification, submits scenarios assembled from the request facts, runs boolean and generative queries, and parses the reasoner's responses into structured decisions. The Reasoner Connector is

designed as a *modular adapter*: it embeds no policy logic of its own and exposes a stable internal interface, so that either side can be replaced independently. On the storage side, the underlying Policy Store implementation can be swapped for a different back-end without affecting the rest of the enforcer. On the reasoning side, the Policy Reasoner itself can be substituted with a different decision engine entirely, provided it honours the same query interface. This two-directional modularity keeps the rest of the enforcer agnostic to both policy-storage and policy-evaluation technology choices (F6).

Session Manager. Tracks approved data-exchange operations as *sessions*, each following a pre / ongoing / revoked / terminated lifecycle inspired by UCON+'s Session Manager role [13]. For every session, it records the current state, the request context, and the agreement-policy version (as maintained by the Policy Repository) under which the session was approved. This allows the enforcer to determine which sessions are affected by a given policy or attribute change and to drive the per-session re-evaluation that realises continuous authorisation (F3, F4).

Obligation Manager. Inspired by the UCON+ role of the same name [13], the Obligation Manager records the obligations and duties produced by the reasoner.

Audit Logger. For each evaluation, the Audit Logger persists the request facts, the agreement-policy version, and the reasoner's decision. The Audit Logger will also keep track of the fulfilled processing activities (F7).

eFLINT Reasoner Interface

The contract between the Policy Enforcer and the eFLINT reasoner determines what a policy author can express without code changes (F6). It is structured as three concentric layers (Figure 5.3), modelled on the brane-chk pattern analysed in chapter 4.

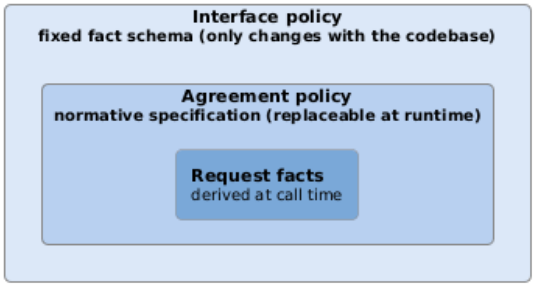

Figure 5.3: Three-layer separation of policy concerns

Layer 1: Interface policy. A small, stable eFLINT specification shipped with the enforcer. It declares the base fact types that mirror the platform's agreement vocabulary (e.g. `requester`, `data-steward`, `archetype`) and five *query facts* that the runtime invokes: overall request admissibility, per-steward admissibility, valid archetypes, valid compute providers, and permitted continuation for in-flight sessions. These query facts have no derivation rules in Layer 1; their `Holds when` clauses are supplied by Layer 2

via eFLINT's `Extend Fact` mechanism. The interface policy also defines the request lifecycle through normative concepts – an act and a duty – providing a formal basis for auditability (F7). The full declarations are given in section 6.3.

Layer 2: Agreement policy. The runtime-mutable normative specification authored by a policy expert. It must supply derivation rules for each Layer-1 query fact. Because the interface policy fixes the query schema, the agreement policy can be rewritten without touching the enforcer code (F2). In practice, Layer 2 decomposes into *shared agreement rules* (intermediate facts and `Extend Fact` clauses shared across a consortium) and *per-steward agreements* (concrete fact instances for each data steward). Section 6.3 walks through the derivation rules step by step.

Layer 3: Request facts. Ephemeral facts generated by the Request Context Builder for a single evaluation. The decision and its derivation tree are persisted by the Audit Logger. Forward-looking attributes such as *purpose* [3] fit in this layer once the corresponding interface-policy fact types are introduced. Section 6.3 demonstrates the full evaluation cycle end-to-end.

This three-layer separation embodies the principle articulated by Binsbergen et al. [3]: *"By separating the rules formalising the norms from the implementation of a system, the transparency, adaptability and maintainability of the system are increased."* It also gives a precise meaning to F2: a runtime policy update is a replacement of Layer 2 only, and is admissible only if the new agreement policy still derives all five Layer-1 query facts. The Policy Repository validates this property by submitting a synthetic probe scenario to the reasoner before activating a new agreement policy.

5.3 Dynamic Policy Update Mechanism (RQ2)

This section describes how the architecture handles runtime policy changes and their effect on in-flight data-sharing operations, addressing F3 and F4. The concrete protocol and implementation are detailed in chapter 6.

Runtime Policy Change Propagation

Figure 5.5 shows the propagation of a policy change end-to-end. The Policy Enforcer is the authoritative driver of the entire workflow: it validates the incoming policy, identifies affected sessions, and produces per-session decisions. The Context Handler only receives a pre-classified batch and acts on it.

The flow consists of four phases. First, the Policy Enforcer validates the incoming agreement policy against the interface policy by running a synthetic probe scenario; if any Layer-1 query fact cannot be derived, the update is rejected (N4). Second, the validated policy is persisted with a fresh version identifier. Third, the Session Manager identifies all ongoing sessions and the Policy Enforcer re-evaluates each one against the new policy using the `permitted-continuation` query fact. Each session's state is updated and the decision is audit-logged. Fourth, a single batched notification is sent to the Context Handler, which acts on the pre-classified per-session decisions without performing any

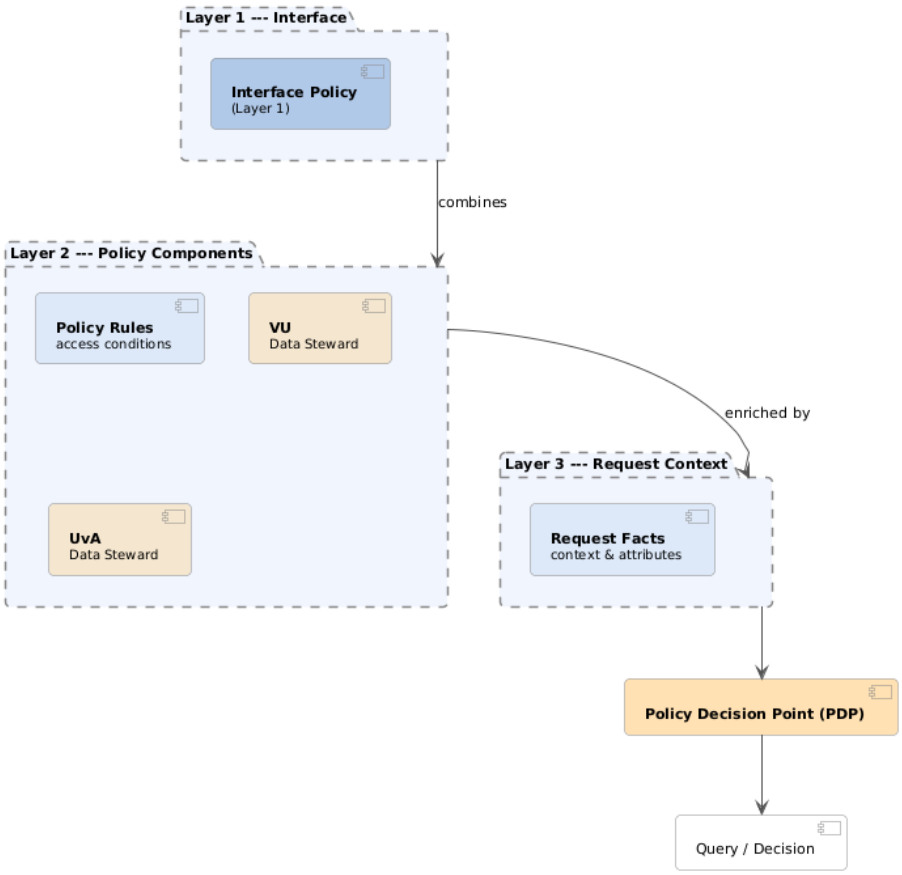

Figure 5.4: Layered policy architecture

policy logic of its own (F4). Between explicit update events, the controller-side PEP continues to consult the Session Manager on a per-call basis, ensuring that recorded decisions take effect at the next policy-relevant operation.

Sources of mutability. UCON identifies three triggers for re-evaluation [22]: changes to subject attributes, changes to resource attributes, and changes to the normative specification itself. The architecture treats all three uniformly: because subject and resource attributes enter the reasoner as request facts, an attribute change reduces to the same re-evaluation loop as a policy change, with the new attribute values substituted into the stored request facts. This uniform treatment allows continuity (F3) to be defined once at the architectural level rather than once per source of change.

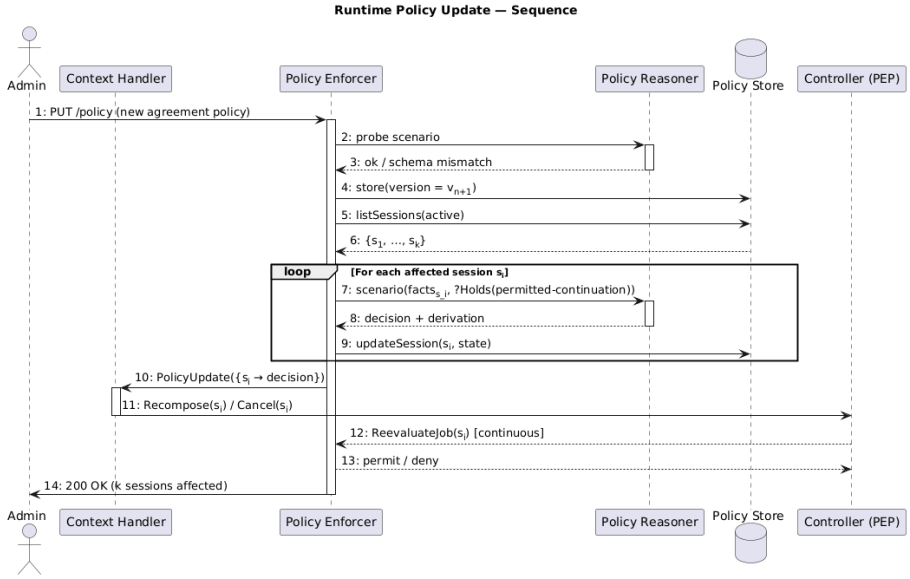

Figure 5.5: Sequence diagram of the runtime policy update mechanism. The Policy Enforcer is the authoritative driver of the workflow: it validates the new agreement policy, identifies affected sessions itself, and produces per-session decisions. The Context Handler only receives a pre-classified batch and acts on it.

Effect on Execution Pathways

A policy update can affect an in-flight execution pathway in three ways, determined by the `valid-archetype` query fact:

No-op. The set of permitted archetypes is unchanged; the existing microservice chain continues.

Recomposition. The permitted archetype set has changed, but is non-empty; the Context Handler selects a new archetype, sends an updated composition request, and stores the updated session.

Revocation. No permitted archetypes remain; the session is moved to revoked and the job is terminated, satisfying UCON+ revocation semantics.

This three-way classification keeps the impact proportional: most granular policy updates affect only a subset of sessions, leave their archetypes unchanged, and therefore produce no execution-pathway side effects.

Chapter 6

Implementation

This chapter describes the proof-of-concept implementation that realises the design of chapter 5. Section 6.1 scopes the prototype; section 6.2 describes the Policy Enforcer components; section 6.3 specifies the eFLINT policy layers.

6.1 Proof-of-Concept Overview

The prototype implements the architecture of section 5.2 as a replacement for the mock policy enforcer analysed in section 4.1. The following capabilities are in scope:

- Dual-strategy validation: eFLINT-based normative reasoning and legacy JSON agreement evaluation, selectable per provider.
- A pre-started reasoner instance pool for concurrent eFLINT evaluations.
- An HTTP debug API for policy experts to query allowed clauses and validate requests outside the normal RabbitMQ flow.
- Session tracking in etcd for ongoing authorisation during policy updates.
- Audit-log record creation after every decision.
- A backward-compatible `ValidationResponse` protobuf format (N2).

The following items are deliberately out of scope:

- Active obligation enforcement: the Obligation Manager is present as a placeholder only (F5).
- Continuous re-evaluation triggered by attribute changes (F3): DYNAMOS does not yet expose an attribute-monitoring hook.
- The `permitted-continuation` derivation rule (commented out in Layer 2).
- Multi-reasoner backends: only eFLINT is supported as a PDP.

6.2 Policy Enforcer Components

Internal Architecture

The shortcomings identified in section 4.1—monolithic request handling, implicit policy logic, and the absence of a formal PDP—motivate a decomposition into distinct, collaborating components. Figure 6.1 shows the resulting component decomposition; the following descriptions walk through each component and note, where applicable, the corresponding XACML role (N1).

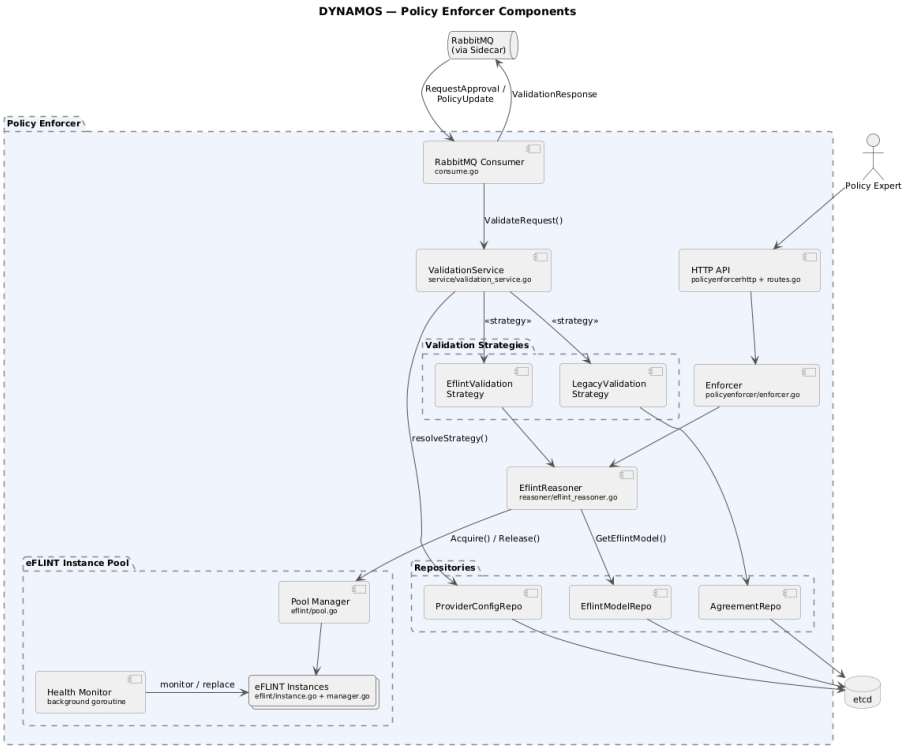

Figure 6.1: Internal component architecture of the redesigned Policy Enforcer.

RabbitMQ Consumer. Routes incoming `RequestApproval` and `PolicyUpdate` protobuf messages to the `ValidationService`; publishes the resulting `ValidationResponse` back to the orchestrator queue.

ValidationService (PEP). Orchestrates the validation of a request: launches concurrent per-provider goroutines, calls `resolveStrategy()` to select a validation strategy per provider, collects results, and assembles the final `ValidationResponse`.

LegacyValidationStrategy. Validates a provider against JSON agreement documents stored in etcd. Preserves the original mock behaviour (N2).

EflintValidationStrategy. Delegates validation to the `EflintReasoner`, which performs normative reasoning via the eFLINT instance pool.

EflintReasoner (PDP/PIP). Implements the `Reasoner` interface. Acquires an idle pool instance, loads the organisation's eFLINT model from etcd via the `EflintModelRepository`, translates agreements and request data into eFLINT facts (Layers 2 and 3), issues queries, parses responses, and releases the instance. Combines the PDP (reasoning) and PIP (fact translation) roles.

Enforcer (HTTP API). Wraps a `Reasoner` and exposes reasoner-agnostic HTTP endpoints for policy experts to query allowed clauses and validate requests outside the RabbitMQ flow.

Repositories (PAP). Three repository interfaces abstract etcd access: `AgreementRepository` (legacy JSON agreements), `EflintModelRepository` (eFLINT policy models), and `ProviderConfigRepository` (per-provider strategy configuration).

eFLINT Instance Pool (PDP). A pool of pre-started eFLINT server processes, each on a unique TCP port. Includes a health monitor goroutine that replaces unhealthy instances and enforces the target pool size; described further in section 6.2.

Validation Workflow

The public protobuf interface is unchanged: the Policy Enforcer receives a `RequestApproval` message via RabbitMQ and returns a `ValidationResponse` (N2). Only the internal evaluation flow differs. Figure 6.2 shows the sequence within the enforcer.

The workflow proceeds in seven steps:

1. **Fetch and translate agreements.** The Request Context Builder retrieves each steward's agreement from etcd and converts it to eFLINT phrases (Layer 2 instances).
2. **Populate Layer 3 facts.** The requester identity and requested stewards are asserted as request facts.
3. **Overall admissibility.** The Reasoner Connector queries `permitted-request` to short-circuit with a deny if no steward is reachable.
4. **Per-steward queries.** For each requested steward, `permitted-at-steward`, `valid-archetype`, and `valid-compute-provider` are queried.
5. **Act trigger.** The `submit-data-request` act is fired for each valid steward, creating `data-request` facts and `obligated-log` duties.
6. **Continuation check.** The `permitted-continuation` query verifies that approved sessions may continue under the current policy.
7. **Audit and response.** The Audit Logger persists the derivation tree; the Reasoner Connector marshals query results into a `ValidationResponse`.

Backward Compatibility: Strategy Pattern

To maintain backward compatibility with existing JSON agreements (N2), the enforcer employs a Strategy pattern. A `ValidationStrategy` interface is implemented by two concrete strategies:

The method `resolveStrategy()` inspects the provider's configuration key in etcd (`/policyEnforcer/configs/{provider}`) to select the appropriate strategy. Different providers within the same request may therefore use different strategies.

Strategy	Agreement format	Data source
`LegacyValidationStrategy`	JSON	`/policyEnforcer/agreements/{steward}`
`EflintValidationStrategy`	eFLINT	Reasoner instance pool

Table 6.1: Validation strategies and their agreement formats.

Table 6.2 lists the etcd key paths used by the Policy Enforcer and related DYNAMOS components.

Path	Purpose	Data format	Used by
`/policyEnforcer/agreements/{steward}`	Legacy JSON agreement	JSON (`api.Agreement`)	`LegacyValidationStrategy`
`/policyEnforcer/eflintModels/{provider}`	eFLINT policy model	Plain text (`.eflint`)	`EflintReasoner`
`/policyEnforcer/configs/{provider}`	Provider config	JSON (`api.ProviderValidationConfig`)	`ValidationService`
`/policyEnforcer/eflintStates/{provider}`	Saved eFLINT states	JSON	State management
`/agents/online/{name}`	Online agent status	JSON	Orchestrator
`/archetypes/{name}`	Archetype configs	JSON	Orchestrator
`/agents/jobs/{agent}/{user}/{job}`	Active job session	JSON (`api.CompositionRequest`)	Orchestrator, Agent
`/agents/jobs/{agent}/queueInfo/{localJob}`	Transient queue entry	Plain text	Agent

Table 6.2: Etcd key paths relevant to the Policy Enforcer.

Reasoner Instance Pool

The eFLINT instance pool manages a set of pre-started eFLINT server processes. Each instance runs on a unique TCP port and is bootstrapped at startup with the Layer 1 interface policy (`01_interface_policy.eflint`), which defines the shared vocabulary and query-fact declarations. When a validation request arrives, an idle instance is acquired; the Layer 2 agreement rules (global agreement file and steward-specific agreement facts) and the per-request Layer 3 facts are then loaded via `SendPhrases`, the relevant queries are issued, and the instance is restarted back to its Layer 1 baseline before being returned to the pool.

Table 6.3 summarises the pool lifecycle across three phases.

Each pool entry (`PoolEntry`) can be in one of three states, listed in table 6.4.

A background goroutine monitors pool health at a ten-second interval. It checks each instance's TCP connection, replaces instances that are unresponsive or have crashed, and enforces the target pool size. If all instances become unhealthy, the enforcer responds with HTTP 503, implementing deny-by-default (N4).

Startup	Validation request	After validation
1. Create pool with target size. 2. Start N eFLINT servers in parallel. 3. Bootstrap each with the interface policy (`01_interface_policy.eflint`). 4. All instances start as `idle`. 5. Start health monitor.	1. `Acquire()` returns an idle instance (blocks up to `AcquireTimeout`). 2. Mark as `in_use`. 3. Load Layer 2 agreement rules and steward facts (`SendPhrases`). 4. Load Layer 3 request facts (`SendPhrases`). 5. Query facts. 6. Parse and filter results.	1. Restart eFLINT server process with interface policy model. 2. If restart fails, mark `unhealthy` (health monitor replaces it). 3. Mark as `idle`. 4. Return to pool (async goroutine).

Table 6.3: Reasoner instance pool lifecycle.

State	Description
`idle`	Available for acquisition; instance has the Layer 1 interface policy loaded.
`in_use`	Currently acquired by a validation goroutine.
`unhealthy`	Process crashed or health check failed; will be replaced.

Table 6.4: Reasoner instance states.

HTTP API

The Policy Enforcer exposes a two-tier HTTP surface alongside the existing RabbitMQ interface.

Reasoner-agnostic endpoints. These endpoints provide a reasoner-independent interface for policy experts to query and debug the enforcer. Each request follows the same pool lifecycle as the RabbitMQ validation flow: the enforcer acquires an idle instance, loads the organisation's model, executes the query, and releases the instance.

Method	Path	Description
`GET`	`/api/v1/policy-enforcer/allowed-clauses`	All allowed clauses (request types, data sets, archetypes, compute providers) for a given organisation and requester.
`POST`	`/api/v1/policy-enforcer/validate`	Validate a full data access request.

Table 6.5: Reasoner-agnostic Policy Enforcer endpoints.

eFLINT pool-management endpoints. These endpoints manage the eFLINT instance pool used for concurrent validation.

Method	Path	Description
GET	/api/v1/eflint/instances	List all pool instances with their state.
GET	/api/v1/eflint/instances/pool-size	Pool size statistics (total, idle, in_use, unhealthy).
PUT	/api/v1/eflint/instances/pool-size	Dynamically resize the pool.

Table 6.6: eFLINT pool-management endpoints.

eFLINT instance-management endpoints. These endpoints manage individual eFLINT server instances, providing direct access to the eFLINT server protocol (sending commands, phrases, querying facts and enabled acts)[1]. They are primarily used for debugging and development.

Method	Path	Description
GET	/api/v1/eflint/status	Instance status (optional ?instance_id=X for pool instances).
POST	/api/v1/eflint/start	Start an eFLINT instance with a model (supports force flag).
POST	/api/v1/eflint/stop	Stop a running instance.
POST	/api/v1/eflint/restart	Restart an instance with its current model.
POST	/api/v1/eflint/command	Send a raw command to the eFLINT server.

Table 6.7: eFLINT instance-management endpoints.

Policy Update and Dynamic Re-evaluation

Policy updates reuse the existing PolicyUpdate protobuf message. Internally, the enforcer converts a policy-update message into a RequestApproval and re-validates all affected sessions, so the same validation pipeline handles both initial requests and re-evaluations.

The Session Manager retrieves active sessions from etcd at /agents/jobs/{agent}/{user}/{job}. For each stored composition request, the enforcer queries permitted-continuation to determine whether the session may continue under the updated policy. Sessions that no longer pass are flagged for termination. Figure 6.3 shows the sequence.

Ongoing authorisation triggered by attribute changes (F3) is not implemented: DYNAMOS does not yet expose attribute-change hooks. The permitted-continuation query provides the mechanism; once such hooks exist, continuous re-evaluation can be activated without changes to the enforcer (future work).

[1] See https://gitlab.com/eflint/haskell-implementation for the eFLINT server specification.

Audit Logging

After every decision, the Audit Logger persists the request facts, policy version, decision outcome, and—where available—the eFLINT derivation tree to etcd. These records satisfy the three auditability conditions of F7: lawfulness is established before processing, the manner of establishment (the derivation) is recorded, and the processing itself is logged. The `obligated-log` duty created by the `submit-data-request` act (section 6.3) provides the normative trigger for this logging step.

6.3 eFLINT Specification

The eFLINT specification realises the three-layer separation introduced in section 5.2. Layer 1 (the interface policy) fixes the vocabulary and query facts; Layer 2 (the agreement rules) supplies the derivation rules that make those query facts hold; Layer 3 (request facts) is generated per call by the Request Context Builder. The full source of Layers 1 and 2 is reproduced in chapter A; this section walks through the agreement rules of Layer 2 step by step, showing how each decision point of the existing mock policy enforcer (fig. 4.7) is translated into a normative derivation.

Layer 1: Interface Policy

The interface policy declares the base fact types that mirror the domain vocabulary already present in the DYNAMOS agreement structure (listing 4): `requester`, `data-steward`, `dataset`, `request-type`, `archetype`, and `compute-provider`. It also declares a helper fact `requested-steward` that marks which data stewards are addressed by the current request, corresponding to the iteration over `data_providers` in the `RequestApproval` protobuf message (listing 1).

The query facts declared in Layer 1 correspond to the decision points the DYNAMOS runtime invokes:

- `permitted-request` – overall request admissibility (at least one valid steward).
- `permitted-at-steward` – per-steward admissibility (requester has standing).
- `valid-archetype` – generative query enumerating admissible archetypes per steward.
- `valid-compute-provider` – generative query enumerating admissible compute providers.
- `permitted-continuation` – ongoing authorisation for in-flight sessions.

These facts have *no* derivation rules in Layer 1; they are declared as empty shells whose `Holds when` clauses are supplied by the agreement rules via `Extend Fact`. Additionally, Layer 1 defines the request lifecycle: the act `submit-data-request` and the duty `obligated-log`, embedding the Hohfeldian power-duty relation discussed in section 4.2. Listing 9 shows the query-fact declarations.

```
// Layer 1 — query facts (no derivation rules)
Fact permitted-request
  Identified by requester

Fact permitted-at-steward
  Identified by requester * data-steward

Fact valid-archetype
  Identified by requester * data-steward * archetype

Fact valid-compute-provider
  Identified by requester * data-steward * compute-provider

Fact permitted-continuation
  Identified by requester * data-steward * archetype
```

Listing 9: Layer 1 query-fact declarations from `01_interface_policy.eflint`. No `Holds when` clauses; derivation rules are supplied by Layer 2.

Layer 2: Agreement Rules

The agreement rules file (`02_agreement_rules.eflint`, reproduced in full in chapter A) translates the procedural logic of the mock policy enforcer into declarative normative derivations. The translation follows the activity diagram of fig. 4.7 step by step: each decision diamond in the flow becomes a `Conditioned by` or `Holds when` clause in eFLINT, and each data-assembly step becomes an intermediate fact declaration.

Agreement Existence

The first check in the policy enforcer flow is whether an agreement exists for the requested data steward (the "Agreement exists?" diamond in fig. 4.7). In the mock implementation, this corresponds to querying etcd at `/policyEnforcer/agreements/{steward}` and checking whether a document is returned. In the normative specification, this is captured by a single fact declaration:

```
Fact agreement Identified by data-steward
```

Listing 10: Agreement existence. An instance `agreement(VU)` holds exactly when the data steward VU has a registered agreement.

At runtime, the Request Context Builder (or the PIP, depending on the workflow) populates this fact for each steward whose agreement is found in etcd. If no agreement exists, the fact is simply absent and all downstream derivations for that steward fail, which is the normative equivalent of the "Add to InvalidDataproviders" path.

Steward Capabilities

When an agreement exists, the mock enforcer reads the top-level `archetypes` and `computeProviders` fields from the JSON document (listing 4). These represent what the data steward supports in general, independent of any specific requester. Two intermediate facts model this:

```
Fact steward-supports-archetype Identified by data-steward * archetype
  Conditioned by agreement(data-steward)

Fact steward-supports-compute-provider Identified by data-steward * compute-provider
  Conditioned by agreement(data-steward)
```

Listing 11: Steward capabilities. The `Conditioned by` clause encodes that these facts can only hold if the steward's agreement exists.

The `Conditioned by` clause is the normative equivalent of the procedural guard: any attempt to assert `steward-supports-archetype(VU, computeToData)` without a prior `agreement(VU)` in the knowledge base is rejected by the reasoner.

User Relation Check

The second decision diamond in fig. 4.7 asks "User in relations?". In the JSON agreement, the requesting user's email is present in the `relations` map. The agreement rules model this with a relational fact:

```
Fact has-relation Identified by requester * data-steward
  Conditioned by agreement(data-steward)
```

Listing 12: User relation check. An instance `has-relation(Alice, VU)` holds when the requester Alice is listed in VU's agreement relations.

Relation-Level Permissions

Within a relation, the JSON agreement specifies fine-grained allowances: `requestTypes`, `dataSets`, `allowedArchetypes`, and `allowedComputeProviders`. Each is captured as a separate intermediate fact, all conditioned on the relation itself:

Archetype Intersection

The core logic of the mock policy enforcer is the intersection step: "Match user archetypes with agreement archetypes" in fig. 4.7. Only archetypes that appear in *both* the relation's `allowedArchetypes` and the steward's top-level `archetypes` are considered valid. In the normative specification, this intersection is expressed as a conjunction:

This single derivation rule replaces the procedural intersection loop. The reasoner evaluates it for all combinations of `(requester, data-steward, archetype)` in

```
Fact relation-allows-request-type Identified by requester * data-steward * request-type
  Conditioned by has-relation(requester, data-steward)

Fact relation-allows-dataset Identified by requester * data-steward * dataset
  Conditioned by has-relation(requester, data-steward)

Fact relation-allows-archetype Identified by requester * data-steward * archetype
  Conditioned by has-relation(requester, data-steward)

Fact relation-allows-compute-provider Identified by requester * data-steward * compute-p
  Conditioned by has-relation(requester, data-steward)
```

Listing 13: Relation-level permissions. Each fact type captures one dimension of the allowances stored in the `relations` map of the JSON agreement.

```
Extend Fact valid-archetype
  Holds when relation-allows-archetype(requester, data-steward, archetype)
    && steward-supports-archetype(data-steward, archetype)
```

Listing 14: Archetype intersection. The Layer 1 query fact `valid-archetype` holds exactly when the archetype is permitted by both the requester's relation and the steward's general capabilities.

the current knowledge base, producing the same result set that the mock enforcer computes by iterating over both lists.

Compute-Provider Validation

Analogously, valid compute providers are derived by intersecting the relation's allowances with the steward's supported providers:

```
Extend Fact valid-compute-provider
  Holds when relation-allows-compute-provider(requester, data-steward, compute-provider)
    && steward-supports-compute-provider(data-steward, compute-provider)
```

Listing 15: Compute-provider intersection. Corresponds to the "Store compute providers" step in fig. 4.7.

Per-Steward Admissibility

Combining the agreement existence and relation checks, the mock enforcer determines whether a steward should be added to `valid_dataproviders` or `invalid_dataproviders`. The normative equivalent is the derivation rule for `permitted-at-steward`:

```
Extend Fact permitted-at-steward
  Holds when agreement(data-steward)
    && has-relation(requester, data-steward)
```

Listing 16: Per-steward admissibility. If both conditions hold, the steward belongs in `valid_dataproviders`; otherwise, it falls into `invalid_dataproviders`.

Overall Request Decision

The final decision in the flow ("ValidDataproviders > 0?" in fig. 4.7) sets `request_approved`. In eFLINT, this maps to `permitted-request`, which holds when at least one requested steward is permitted:

```
Extend Fact permitted-request
  Holds when requested-steward(data-steward)
    && permitted-at-steward(requester, data-steward)
```

Listing 17: Overall request decision. The fact `permitted-request(Alice)` holds if at least one of the stewards Alice requested passes the per-steward admissibility check.

The reasoner produces `true` for `?Holds(permitted-request(…))` if any requested steward satisfies the conjunction, directly corresponding to setting `request_approved = true` in the `ValidationResponse`.

Summary: Flow Diagram to Normative Derivation

Table 6.8 summarises the correspondence between each step of the policy enforcer activity diagram (fig. 4.7) and the eFLINT fact or rule that captures it.

Flow step	eFLINT construct	Listing
Agreement exists?	`agreement(data-steward)`	10
Steward archetypes / providers	`steward-supports-*`	11
User in relations?	`has-relation(req, steward)`	12
Relation allowances	`relation-allows-*`	13
Match archetypes	`valid-archetype` (conjunction)	14
Store compute providers	`valid-compute-provider`	15
Add to Valid/InvalidDataproviders	`permitted-at-steward`	16
ValidDataproviders > 0?	`permitted-request`	17

Table 6.8: Mapping from the policy enforcer activity diagram (fig. 4.7) to eFLINT normative constructs in Layer 2.

The normative specification thus provides a declarative, auditable counterpart to every decision branch of the procedural mock, while additionally supporting continuous authorisation for in-flight sessions – a capability absent from the original design.

Scenario Walkthrough: Request Approval

This section demonstrates the full evaluation cycle by walking through a single request-approval scenario. The scenario uses the same domain as the running example from chapter 4: requester *Niels* requests data from two stewards, *VU* and *UVA*, whose agreements were shown in listing 4. The equivalent `RequestApproval` protobuf message (listing 1) would be:

```
RequestApproval {
  type: "requestApproval"
  user: { id: "GUID", user_name: "Niels" }
  data_providers: ["VU", "UVA"]
  destination_queue: "policyEnforcer-in"
}
```

Listing 18: Example `RequestApproval` message triggering the scenario walkthrough.

Step 1: Populate Request Facts

The Request Context Builder translates the incoming protobuf into eFLINT request facts (Layer 3). The requester identity and the list of requested stewards are the only inputs derived from the `RequestApproval` message itself; the agreement facts for each steward (Layer 2 instances) are loaded from etcd at request time, after the idle instance,already seeded with the Layer 1 interface policy, is acquired.

```
+requester(Niels).
+requested-steward(VU).
+requested-steward(UVA).
```

Listing 19: Request facts generated by the Request Context Builder for this scenario.

Step 2: Overall Admissibility Query

The eFLINT Connector issues the top-level boolean query to determine whether the request is admissible at all:

```
?Holds(permitted-request(Niels)).
// Response: query successful
```

Listing 20: Overall admissibility query. The fact holds because at least one requested steward (VU, UVA, or both) has a valid relation with Niels.

This corresponds to the “ValidDataproviders > 0?” check at the end of fig. 4.7, but is evaluated up front so the enforcer can short-circuit with a deny response if no steward is reachable.

Step 3: Per-Steward Admissibility

For each requested steward, the connector queries whether the requester has standing:

```
?Holds(permitted-at-steward(Niels, VU)).
// Response: query successful

?Holds(permitted-at-steward(Niels, UVA)).
// Response: query successful
```

Listing 21: Per-steward admissibility queries. Both stewards pass because Niels has a relation in each agreement.

A steward for which `permitted-at-steward` does not hold is placed in `invalid_dataproviders` in the response.

Step 4: Archetype Discovery

For each admitted steward, the connector issues a generative query to enumerate the valid archetypes:

```
?-archetype When valid-archetype(Niels, VU, archetype).
// Response: archetype("ComputeToData")

?-archetype When valid-archetype(Niels, UVA, archetype).
// Response: archetype("ComputeToData")
//           archetype("DataThroughTtp")
```

Listing 22: Generative archetype queries. VU yields one archetype (the intersection of Niels's relation allowing only `ComputeToData` with VU's supported set); UVA yields two.

This replaces the "Match user archetypes with agreement archetypes" loop in the mock. The difference for UVA arises because Niels's relation at UVA allows both `ComputeToData` and `DataThroughTtp`, and UVA supports all three archetypes at the steward level; the intersection yields two.

Step 5: Compute-Provider Discovery

Similarly, compute providers are enumerated per steward:

Step 6: Action Trigger and Duty Creation

After a positive decision, the eFLINT Connector fires the `submit-data-request` act for each valid steward. This act is enabled because its `Holds when` clause references `requested-steward` and `permitted-at-steward`, both of which hold. Firing the act creates the `data-request` fact and the `obligated-log` duty:

```
?-compute-provider When valid-compute-provider(Niels, VU, compute-provider).
// Response: compute-provider("SURF")

?-compute-provider When valid-compute-provider(Niels, UVA, compute-provider).
// Response: compute-provider("SURF")
```

Listing 23: Generative compute-provider queries. Both stewards yield SURF as the only valid provider.

```
submit-data-request(Niels, VU).
// Response: +data-request(requester("Niels"),data-steward("VU"))
// +obligated-log(data-steward("VU"),requester("Niels"))

submit-data-request(Niels, UVA).
// Response: +data-request(requester("Niels"),data-steward("UVA"))
// +obligated-log(data-steward("UVA"),requester("Niels"))

?Holds(data-request(Niels, VU)).
// Response: query successful

?Holds(data-request(Niels, UVA)).
// Response: query successful
```

Listing 24: Triggering the submission act. The `data-request` facts record the approved exchange; the `obligated-log` duty is created as a side effect (auditable by the Audit Logger).

The `obligated-log` duty embodies the Hohfeldian power-duty relation: by exercising the power to submit a data request, the requester places the data steward under an obligation to log the processing of their data. In the current proof-of-concept, this duty is recorded by the Audit Logger; active enforcement awaits the Obligation Manager, discussed as future work.

Step 7: Continuation Verification

Finally, the Session Manager verifies that the established sessions pass the continuation check under the current agreement policy. This query will later be re-issued during dynamic policy updates (section 5.3):

Assembling the ValidationResponse

The eFLINT Connector marshals the query results into the `ValidationResponse` protobuf (listing 2). Listing 26 shows the resulting message for this scenario:

The key observation is that the `ValidationResponse` is structurally identical to what the mock enforcer would produce for the same input (N2). The difference is twofold: (i) the decision is grounded in a formal normative derivation rather than procedural

```
?Holds(permitted-continuation(Niels, VU, ComputeToData)).
// Response: query successful

?Holds(permitted-continuation(Niels, UVA, ComputeToData)).
// Response: query successful
```

Listing 25: Continuation queries. Confirms that the approved sessions may continue under the current agreement policy.

```
ValidationResponse {
  type: "validationResponse"
  request_type: "sqlDataRequest"
  valid_dataproviders: {
    "VU": {
      archetypes: ["computeToData"]
      compute_providers: ["SURF"]
    }
    "UVA": {
      archetypes: ["computeToData", "dataThroughTtp"]
      compute_providers: ["SURF"]
    }
  }
  invalid_dataproviders: []
  auth: { access_token: "...", refresh_token: "..." }
  user: { id: "GUID", user_name: "Niels" }
  request_approved: true
  valid_archetypes: {
    user_name: "Niels"
    archetypes_per_steward: {
      "VU": ["computeToData"]
      "UVA": ["computeToData", "dataThroughTtp"]
    }
  }
}
```

Listing 26: Resulting `ValidationResponse` assembled from the eFLINT query results. The structure is identical to what the mock enforcer would produce for the same input; the difference is that the decision is now backed by a formal derivation tree.

intersection logic, and (ii) the Audit Logger can persist the derivation tree alongside the decision (F7), providing a machine-verifiable justification for every approval or denial.

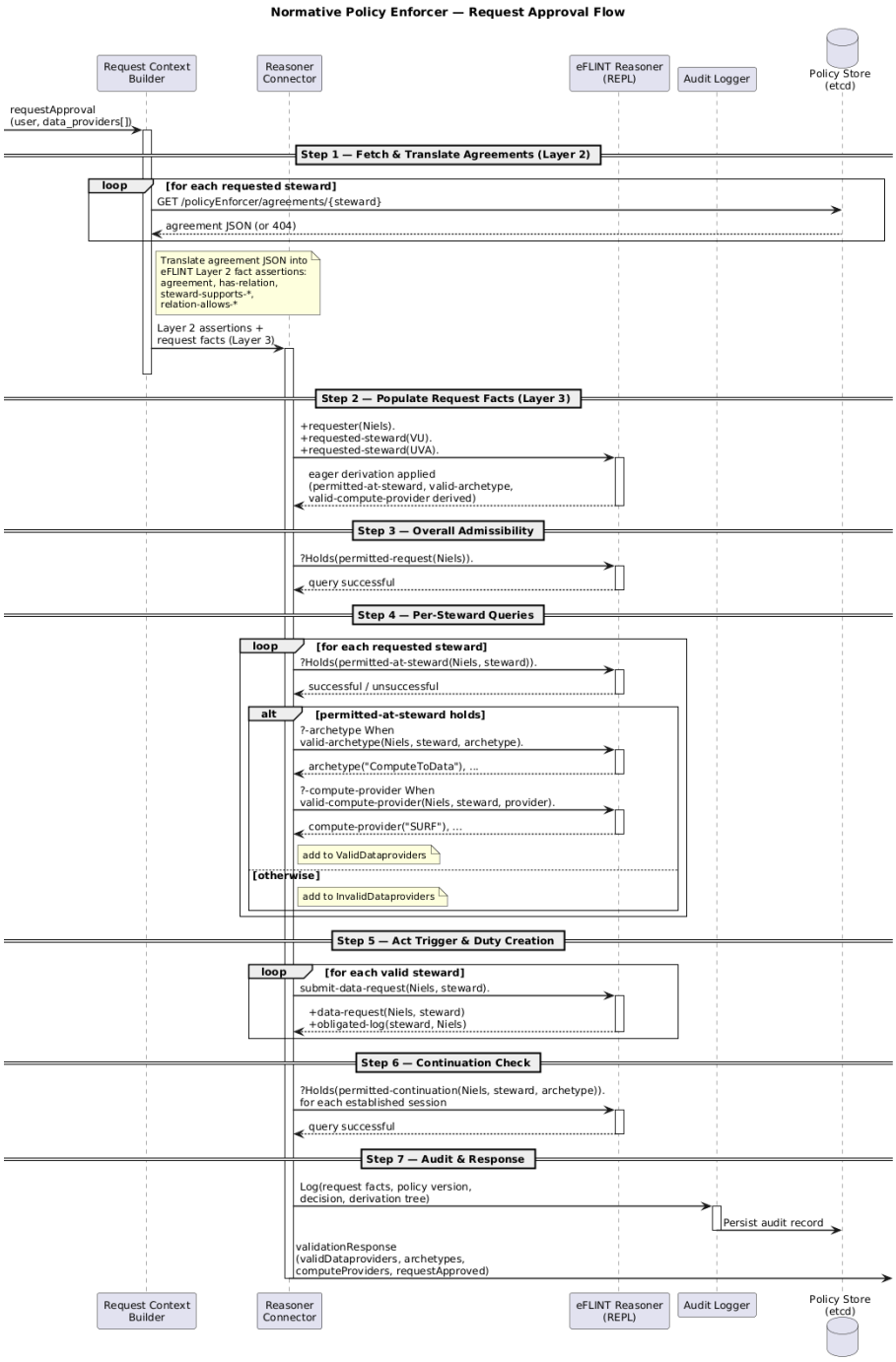

Figure 6.2: Validation workflow within the Policy Enforcer.

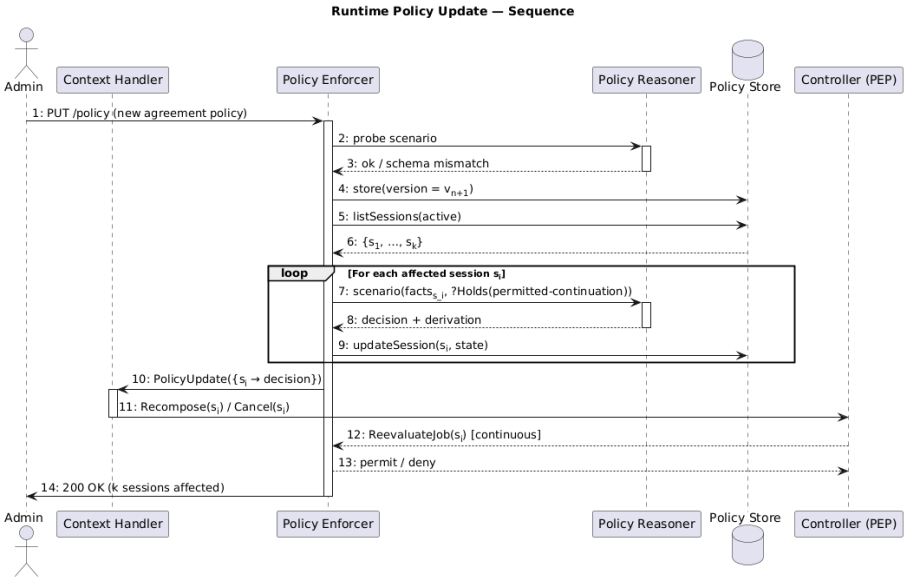

Figure 6.3: Runtime policy update and session re-evaluation sequence.

Chapter 7

Evaluation

This chapter evaluates the proof-of-concept Policy Enforcer of chapter 6 through *correctness testing* of the request-approval pathway, establishing whether the implementation behaves as prescribed by the normative specification of chapter 5 (SRQ3).

Section 7.1 establishes the overall evaluation strategy and clarifies the role of the eFLINT reasoner as the oracle for differential testing. Section 7.2 presents the correctness evaluation.

7.1 Evaluation Strategy

The evaluation centres on *correctness*: whether, for a given input, the implementation generates the response dictated by the normative eFLINT specification. Correctness is established through differential testing, using the eFLINT reasoner as an oracle, targeting the request-approval pathway (SRQ3).

Correctness via differential testing against the eFLINT reasoner. The eFLINT reasoner serves as the *oracle*: it evaluates the same eFLINT scenario file that the human reviewer reads – a file that is also self-verifying through `/#violated` annotations checked by the eFLINT unit-tester – and the predicate hold-results are projected into the `ValidationResponse` shape returned by the Policy Enforcer. The Policy Enforcer is then driven with the HTTP request that the same scenario denotes, and the captured response is compared field-by-field against the oracle's projection. Because the reasoner is shared by both sides of the comparison, the test does not validate the normative specification itself; it validates the integration plumbing – the Request Context Builder, the eFLINT Connector, and the response marshalling – which is exactly the surface that this thesis introduces (N1, F5).

Reproducibility. All scenarios used in the correctness evaluation are maintained in the evaluation repository[1] together with the Python/pytest test harness and the per-scenario YAML manifests. The full harness output from the final run is reproduced in chapter B.

[1] https://github.com/nielsarts/dynamos-eflint-test

7.2 Correctness of the Request-Approval Implementation

This section establishes that, for the defined set of request-approval scenarios, the `ValidationResponse` produced by the implemented Policy Enforcer is equivalent to the response specified by the normative specification. This orientation is the primary correctness test for the prototype because the request-approval pathway is the only one every data-sharing operation traverses; all other orientations depend on it.

Goal and Hypothesis Schema

The goal is operationalised as a single hypothesis schema, instantiated once per scenario:

> H_i: For scenario S_i, the `ValidationResponse` returned by the Policy Enforcer when invoked with the protobuf input denoted by S_i is equivalent (in the sense of section 7.2) to the `ValidationResponse` projected from the eFLINT reasoner's output on S_i.

Each scenario S_i documents its expected query outcomes through the `/#assert` and `/#violated` annotations embedded in the `.eflint` file (available in the evaluation repository); these annotations are the human-readable form of H_i. The machine-readable form is the in-memory `ExpectedValidationResponse` projected by the test-time oracle (section 7.2) and cross-checked against the `HYPOTHESIS_TABLE`.

Oracle: the eFLINT Reasoner

The oracle is the unmodified `eflint-repl` binary, driven at test time by the Python `harness.oracle` module (`derive_expected`) that:

1. flattens the three-file fixture bundle (`01_interface_policy.eflint`, `02_agreement_rules.eflint`, and the relevant `agreement_<steward>.eflint` files) in-process;
2. plants the Layer-3 request facts (`+requester(...)`, `+requested-steward(...)`) derived programmatically from the manifest's `request` block;
3. for each predicate in table 7.1, submits an `Invariant q_ When pred(…).` phrase and checks whether the reply contains `violated invariant!: q_` – presence indicates the predicate does *not* hold, absence indicates it holds;
4. collects the boolean hold-results into an in-memory `ExpectedValidationResponse` object shaped like the `ValidationResponse` returned by the SUT.

The oracle result is computed fresh for each test run and stored in memory; no `expected.json` file is persisted. As an independent cross-check, the `HYPOTHESIS_TABLE` in the pytest suite records the hand-derived expected outcome for every scenario; the test `test_oracle_matches_hypothesis` asserts that the live oracle projection agrees with this hand-coded table, catching oracle regressions before any SUT call is made.

eFLINT predicate probed	ValidationResponse field
`permitted-request(`r`)`	`request_approved`
`permitted-at-steward(`r`, `s`)`	$s \in$ `valid_dataproviders` (holds) or $s \in$ `invalid_dataproviders` (violated)
`valid-archetype(`r`, `s`, `a`)`	`valid_dataproviders[`s`].archetypes` (enumerated per candidate a)
`valid-compute-provider(`r`, `s`, `p`)`	`valid_dataproviders[`s`].compute_providers` (enumerated per candidate p)

Table 7.1: Projection from eFLINT predicate outcomes to `ValidationResponse` fields. Each predicate is probed via `Invariant q When pred(…).` and the violation flag. The same mapping is applied symmetrically by the eFLINT Connector inside the Policy Enforcer.

Scenario files as self-verifying specifications. The scenario `.eflint` files serve a dual role: they are human-readable normative descriptions of each test case *and* machine-verified via the eFLINT unit-tester (`eflint-test.py`)[2] in the consistency gate. The unit tester recognises `/#violated` annotations embedded in the file and verifies them at parse time; any unsatisfied annotation causes the scenario to be reported as inconsistent before the SUT is ever called. There is, however, a deliberate syntactic difference between the way the *Policy Enforcer* queries the reasoner and the way the *scenario file* encodes the same assertion. The Policy Enforcer submits a direct `?Holds` query:

```
?Holds(permitted-request(Niels)).
```

The scenario file expresses the same assertion as a named invariant with a negated `/#violated` guard so that the unit-tester can check it:

```
Invariant scenario_one_permitted_request
  When permitted-request("Niels").
//#violated !scenario_one_permitted_request.
```

Both forms ask the same question – *does `permitted-request("Niels")` hold?* – but via different eFLINT mechanisms. The `Invariant`/`//#violated` idiom is the standard pattern supported by `eflint-test.py`; the harness oracle adopts the same idiom for consistency. Because the oracle and the scenario annotations use the same underlying reasoner and the same predicates, any difference between them would indicate either a malformed scenario or a broken reasoner, not a defect in the Policy Enforcer.

System Under Test

The system under test is the Policy Enforcer of section 6.2 as deployed inside DYNAMOS, exposed via the HTTP endpoint `POST /api/v1/policy-enforcer/validate`. The

[2] eFLINT unit-tester: https://gitlab.com/eflint/tools/unit-tester. The tool is a Python wrapper around `eflint-repl` that processes annotation comments (`/#violated`, `/#assert`) embedded in eFLINT files to define expected and unexpected output.

harness calls this endpoint directly (behind the nginx ingress controller that is already bound to `localhost:80`); no API Gateway or orchestrator hop is on the critical path of the test. This keeps the test surface focused on the Policy Enforcer's own correctness: marshalling, eFLINT Connector, and response projection.

Test Scenarios

Table 7.2 lists the scenarios used in this orientation. They are taken from `eflint/scenario_01_approved.eflint` and the four split scenario files `scenario_02a` through `scenario_04b` and cover four behavioural classes: full approval (S1), partial approval (S2), denial by absence of agreement and absence of relations (S3a, S3b), and denial by mismatched steward and restricted archetype (S4a, S4b). The continuation scenarios (`scenario_05a` through `scenario_05c`) target the continuous-authorisation orientation and are excluded here.

ID	Scenario	Hypothesis (informal)
S1	Approved Niels full	*approved* VU: archetypes {ComputeToData}, compute {SURF} UVA: archetypes {ComputeToData, DataThroughTtp}, compute {SURF}
S2a	Approved Alice UVA	*approved* UVA: archetypes {DataThroughTtp}, compute {SURF}
S2b	Approved Alice partial	*approved* (partial) valid — UVA: archetypes {DataThroughTtp}, compute {SURF} invalid — VU
S3a	Denied Bob no agreement	*denied* (no agreement) invalid — SURF_org
S3b	Denied Bob no relation	*denied* (no relations) invalid — VU, UVA
S4a	Denied Alice wrong steward	*denied* (no relation at VU) invalid — VU
S4b	Denied Alice restricted archetype	*approved* (restricted) UVA: archetypes {DataThroughTtp} only ComputeToData, ReproducableScience explicitly denied

Table 7.2: Request-approval scenarios. The `/#assert` / `/#violated` annotations in each scenario file are the source of the informal hypothesis text; the `HYPOTHESIS_TABLE` in the pytest suite records the same outcomes hand-coded for independent oracle cross-checking.

Test Artefacts

For each scenario, the test harness consumes two files, both stored in the harness repository under `dynamos-test-scenarios/`:

`dynamos-test-scenarios/scenario_<id>.eflint.` The eFLINT scenario file. It declares the per-scenario request facts and carries `/#violated` / `/#assert` annotations that the consistency gate verifies before the SUT is called.

`dynamos-test-scenarios/manifests/scenario_<id>.yaml.` The YAML manifest. It specifies the eFLINT file set to load (interface policy, shared rules, per-steward agreements, and the scenario file), the HTTP request body to send to the SUT, the

candidate archetype/compute-provider universe for oracle enumeration, and the assertion flags controlling which fields the comparator checks.

Listing 27 shows the manifest's `request` block for S1; the oracle builds Layer-3 eFLINT facts (`+requester(...)`, `+requested-steward(...)`) from these fields programmatically, and the harness serialises the same block as the HTTP POST body sent to the SUT.

```yaml
request:
  user:
    id: "1"
    user_name: Niels
  data_providers: [VU, UVA]
  type: requestApproval
```

Listing 27: The `request` block in the YAML manifest for scenario S1 (approved, Niels at {VU, UVA}). The harness serialises this as the HTTP POST body sent to `POST /api/v1/policy-enforcer/validate`.

Test Harness

The harness is a Python pytest suite.[3] A full scenario run proceeds in five phases:

1. **Consistency gate.** The eFLINT unit-tester (`eflint-test.py`) is run against the scenario's `.eflint` file. Any `/#violated` or `/#assert` annotation failures abort the run immediately; this guards against a broken oracle producing a false-pass result against the SUT.
2. **Oracle projection.** The oracle (`derive_expected`) runs the `eflint-repl` on the fixture bundle and the programmatically constructed Layer-3 facts, producing an in-memory `ExpectedValidationResponse`.
3. **Etcd seeding.** The shared rules and per-steward agreement files are written into the running DYNAMOS etcd instance so the SUT evaluates against the same agreement state as the oracle.
4. **SUT call.** `POST /api/v1/policy-enforcer/validate` is sent with the manifest's request body; the response is parsed into an `ActualValidationResponse`.
5. **Comparison.** The comparator performs a per-steward diff according to the rules of section 7.2.

Comparison Rules

Two `ValidationResponse` values are considered *equivalent* when:

- `request_approved` is bit-equal;

[3]The test harness and related evaluation code are available at https://github.com/nielsarts/dynamos-eflint-test.

- the keysets of `valid_dataproviders` are equal, and for every key the `archetypes` and `compute_providers` lists are set-equal (order is not significant: the eFLINT `Invariant` enumeration order is implementation-defined);
- `invalid_dataproviders` is set-equal (same reason);
- `auth.access_token` and/or `auth.refresh_token` are present and non-empty when the oracle reports `request_approved == true`. Token *values* are masked in the comparison because they are non-deterministic.

Fields not enumerated above (`user.id`, `user.user_name`, `type`, `options`, and the token values themselves) are not asserted by the comparator; the manifest's `assertions` list governs which fields are checked, and the default set for all approval scenarios is {`request_approved`, `valid_dataproviders`, `invalid_dataproviders`, `auth_present_when_approved`}.

Results

ID	Hypothesis	Verdict	Diff (observed ⊖ expected)	Duration
H1	S1 equivalent	pass	—	886 ms
H2a	S2a equivalent	pass	—	573 ms
H2b	S2b equivalent	pass	—	774 ms
H3a	S3a equivalent	pass	—	11 ms
H3b	S3b equivalent	pass	—	669 ms
H4a	S4a equivalent	pass	—	452 ms
H4b	S4b equivalent	pass	—	554 ms

Table 7.3: Request-approval correctness results. All seven hypotheses passed with an empty diff; the full harness output is reproduced verbatim in chapter B. Duration is the wall-clock time from SUT call to parsed response as reported by the harness.

All seven hypotheses H1–H4b were confirmed: the comparator observed no field-level difference between the oracle projection and the SUT response for any of the seven scenarios. The consistency gate (oracle cross-check against the `HYPOTHESIS_TABLE`) passed in every case before the SUT was called, ruling out oracle regressions as a confound.

The duration column reveals a notable outlier: S3a completed in 11 ms, two orders of magnitude faster than the other scenarios. This is expected: when no steward-specific eFLINT model exists in etcd for the requested data provider (`SURF_org`), the eFLINT Connector bypasses normal processing, avoids invoking the eFLINT reasoner, and returns an immediate denial response. The remaining six scenarios, which require a full eFLINT evaluation, took between 452 ms and 886 ms; this variation reflects the time the eFLINT reasoner itself takes to reason, as the Policy Enforcer maintains warm, running instances and does not incur start-up overhead per invocation. These duration figures are incidental observations reported by the test harness; they corroborate that the end-to-end path through the eFLINT Connector is exercised for all scenarios that require it, but they do not constitute a controlled performance measurement. Nevertheless, the observed end-to-end durations, sub-second for all scenarios requiring full eFLINT evaluation, appear acceptable in the context of data-sharing request approval within DYNAMOS, where requests typically initiate longer-lived data transfer operations rather

than short interactive transactions. A formal characterisation of the overhead introduced by the Policy Enforcer, as required by N3, is left as future work.

Threats to Validity (orientation-local)

Construct validity. The eFLINT reasoner is shared by both sides of the differential test. A defect in the reasoner would therefore mask itself; this is mitigated by the `HYPOTHESIS_TABLE` in the pytest suite, which was authored by hand against the reasoner's documented semantics and serves as a redundant independent oracle. The consistency gate (`test_scenario_is_self_consistent`) provides a second safeguard: it verifies each scenario's `/#assert` annotations against `eflint-test.py` before the SUT is ever called.

Internal validity. The harness calls the Policy Enforcer directly over HTTP; etcd seeding ensures both sides operate on the same agreement state. The nginx ingress and the Kubernetes cluster networking are on the critical path but are infrastructure components, not part of the implementation under evaluation.

External validity (scenario coverage). The seven hypotheses cover the four behavioural classes that the mock policy enforcer of chapter 4 also distinguishes; they do not cover every combination of relation, archetype, and compute provider. Combinatorial coverage is left to property-based testing in future work.

Chapter 8

Discussion

This chapter interprets the evaluation results of chapter 7, discusses architectural implications of the design choices made in chapters 5 and 6, answers the research questions in light of the evidence obtained, and reflects on the limitations and future directions of this work.

8.1 Discussion of the Results

The correctness evaluation confirmed that the Policy Enforcer produces decisions equivalent to those predicted by the eFLINT reasoner acting as an oracle across all seven request-approval scenarios. This result establishes that the integration plumbing (the Request Context Builder, the eFLINT Connector, and the response marshalling) faithfully translates between the DYNAMOS protobuf interface and the eFLINT normative specification without introducing semantic distortion.

The differential testing methodology deserves explicit reflection. Because the eFLINT reasoner appears on both sides of the comparison, the evaluation validates the *integration* rather than the *specification*: a defect in the eFLINT specification itself would not be caught. Two independent safeguards mitigate this risk: the hand-authored `HYPOTHESIS_TABLE` that cross-checks the oracle before any SUT call, and the self-verifying scenario annotations checked by the eFLINT unit-tester. Together, these create a layered assurance structure in which specification errors must evade both the human reviewer and the annotation consistency gate simultaneously.

The observed end-to-end durations (452–886 ms for full eFLINT evaluations; 11 ms for the fast-path denial in S3a) indicate that the reasoning overhead is sub-second for the evaluated scenarios. While these figures were incidental and not collected under controlled conditions, they suggest that the eFLINT-based enforcement is compatible with the latency profile of DYNAMOS data-sharing operations, which initiate multi-step orchestration workflows rather than interactive low-latency transactions. The outlier confirms a useful architectural property: when no agreement model exists for a requested provider, the eFLINT Connector short-circuits without invoking the reasoner, providing graceful degradation (N4) with negligible overhead.

8.2 Architectural Implications

Separation of concerns. The decomposition into six internal components, each aligned with an XACML or UCON+ role, corrects the role-boundary violation in the original DYNAMOS design, where the orchestrator selected affected jobs on policy changes rather than the Policy Enforcer (chapter 4). The evaluation provides indirect evidence that this decomposition is effective: the test harness addresses the Policy Enforcer in isolation, without requiring orchestrator cooperation for any decision, confirming that policy logic resides entirely within the enforcer boundary.

Modular adapter pattern. The Reasoner Connector acts as a two-directional modular adapter, decoupling both the policy-store back-end and the reasoning engine from the rest of the enforcer. This modularity is exercised concretely in the dual-strategy implementation: the `EflintValidationStrategy` and `LegacyValidationStrategy` coexist, selectable per provider, demonstrating that the enforcer can accommodate heterogeneous reasoning approaches without architectural changes. The same modularity provides the extension point for the Clingo-based reasoner discussed in section 8.5.

Three-layer specification. The three-layer eFLINT specification (interface policy, agreement rules, request facts) provides a separation between stable query schema and mutable agreement logic. This separation delivers two concrete benefits confirmed by the evaluation: (i) the test harness exercises seven structurally different scenarios without any change to the interface policy or the enforcer code, validating the stable query interface requirement (F6); and (ii) the etcd-seeding step of the test harness demonstrates that agreement policies can be swapped at runtime without restarting any component (F2).

However, the layered approach introduces trade-offs in debuggability and maintainability. Because policy derivations span three files that are concatenated at evaluation time, tracing a derivation failure to its root cause requires the policy author to understand the interaction between all layers, particularly how `Extend Fact` clauses in the agreement rules bridge the fixed interface predicates to consortium-specific terms. This complexity is an inherent consequence of providing a stable interface anchor for the Policy Enforcer to interact with the reasoner: the interface policy must abstract over the diversity of possible agreement policies, which imposes a cognitive cost on authors working at the agreement layer.

Updating the interface policy is a more heavyweight operation than updating agreement rules. By design, the interface policy changes only together with the Policy Enforcer codebase (which depends on the query predicates it defines), meaning an interface-policy update requires a coordinated codebase release rather than a runtime hot-swap. While this stability is intentional (it protects running sessions from query-schema drift), it means that evolving the normative vocabulary requires version management across layers. Specifically, if a lower-level layer, such as the interface policy or the shared agreement rules, is updated, compatibility issues may propagate upward: a steward's agreement policy written against the previous version of the shared rules may fail to derive predicates that the interface policy expects. In a production deployment, this risk

would be mitigated by a policy negotiation process: before a new agreement policy takes effect, the signing parties should chain the relevant policy-layer versions and validate compatibility through a synthetic probe scenario, the same mechanism the enforcer uses internally for update validation (section 5.3).

Centralised enforcement. The intermediary-governed topology, in which a single Policy Enforcer centralises all decisions, provides a clear administration point for runtime policy updates and immediate re-evaluation. The evaluation confirms that this centralisation works correctly for the defined scenarios. However, as discussed in chapter 3, this topology forgoes the decentralised autonomy of self-governed approaches (IDS connectors, BRANE's per-site checker), which may be preferable in consortia where participants require local sovereignty over enforcement decisions.

8.3 Answering the Research Questions

SRQ1: Architectural Integration. The proposed architecture demonstrates that a normative policy enforcer can be integrated into DYNAMOS's microservice middleware by (i) decomposing enforcement into XACML-aligned components, (ii) embedding the eFLINT reasoner as a pooled PDP process behind a modular adapter, and (iii) maintaining a stable query interface that decouples agreement evolution from enforcer code changes. The evaluation confirms that this integration produces correct decisions for the defined scenarios while preserving backward compatibility with existing DYNAMOS interfaces (N2).

SRQ2: Dynamic Agreement Updates. The three-layer specification pattern and the runtime update mechanism show that policy changes can dynamically alter execution by: replacing the agreement layer at runtime, re-evaluating affected sessions via `permitted-continuation`, and producing per-session decisions (no-op, recomposition, or revocation) for the Context Handler. The design localises all re-evaluation logic within the Policy Enforcer (F4), avoiding the separation-of-concerns violation present in the original design. While the implementation covers event-triggered continuity (policy replacement triggers re-evaluation), attribute-triggered continuity awaits DYNAMOS-side monitoring hooks.

SRQ3: Correctness of the Integration. The differential testing evaluation established the correctness of the request-approval pathway across seven scenarios covering four behavioural classes. All hypotheses were confirmed with empty diffs. The evaluation scope is bounded: it validates the integration plumbing against the normative specification but does not validate the specification itself; it does not cover all combinatorial input possibilities; it does not exercise error or fault conditions; and it does not include controlled performance measurement. Within these bounds, the evidence supports the claim that the implemented Policy Enforcer correctly enforces the normative policies defined in the evaluation scenarios.

Main Research Question. Synthesising the sub-question answers: a policy enforcer can be integrated into microservice-based DDM middleware to ensure compliance with mutable policies by replacing procedural enforcement with normative reasoning over a layered specification, providing a runtime update mechanism that re-evaluates affected sessions, and recording derivation trees for auditability. The DYNAMOS prototype demonstrates feasibility; the evaluation confirms correctness within the defined scenario scope.

8.4 Limitations

Absence of controlled performance benchmarking. This thesis did not conduct controlled performance benchmarking of the Policy Enforcer integration. The correctness evaluation produced incidental end-to-end durations of 452–886 ms per request under single-request, non-concurrent conditions, which appear compatible with the latency profile of data-sharing operations in DYNAMOS. However, these figures do not satisfy the bounded overhead requirement (N3): they were not collected under controlled load, do not account for concurrent requests, and omit baseline measurements against the unmodified DYNAMOS middleware.

Scenario coverage. The seven request-approval scenarios cover four behavioural classes (full approval, partial approval, denial by absent agreement/relation, denial by mismatched steward/restricted archetype) but do not exhaustively cover all combinations of relation, archetype, and compute provider. The continuous-authorisation, policy-update, and audit-record orientations were not included in the evaluation scope and are identified as future work. Combinatorial coverage through property-based testing would strengthen confidence in edge-case correctness.

Absence of fault-path and error-condition testing. The evaluation covers only the happy flow: every scenario presents a well-formed request to a fully operational Policy Enforcer and verifies the resulting `ValidationResponse`. No test exercises the error and fault conditions that the implementation is specifically designed to handle. For example, the behaviour when the eFLINT instance pool is exhausted (all instances busy or unhealthy) is not tested, nor is the response when a pool instance crashes mid-evaluation, when the eFLINT reasoner returns a malformed reply, or when etcd is temporarily unavailable during model retrieval. The implementation addresses these cases through deny-by-default (N4): the pool returns HTTP 503 when all instances are unhealthy, and the Reasoner Connector denies the request rather than silently admitting it on any parse or connectivity error. However, without fault-injection testing, it is not confirmed that every failure mode actually triggers a safe denial rather than an unexpected exception or a silent pass. This gap is particularly relevant for N4, which is stated as a requirement, realised in the implementation, but remains unvalidated by the evaluation.

Attribute-triggered continuity. The implementation supports event-triggered continuity (policy updates trigger re-evaluation) but not attribute-triggered continuity

(changes to subject or resource attributes during a session). DYNAMOS does not currently expose attribute-monitoring hooks; until it does, the continuous authorisation guarantee is limited to policy-change events.

Single-reasoner dependency. The prototype supports only the eFLINT reasoner as PDP. While the Reasoner Connector is designed as a modular adapter, no alternative back-end has been validated. A reasoner defect would affect all enforcement decisions simultaneously, unlike a distributed multi-PDP topology.

Obligation enforcement. The Obligation Manager is present only as a placeholder. Duties such as `obligated-log` are created and recorded by the Audit Logger, but no active enforcement mechanism monitors whether obligations are fulfilled within their deadlines or triggers compensatory actions in the event of a violation.

8.5 Future Work

Controlled performance evaluation. Future work should address the N3 gap through controlled experiments measuring request latency and throughput under a range of steady-state loads, benchmarking against a baseline without enforcement, and identifying degradation patterns under stress. The instance pool architecture provides a natural parameter for such experiments: varying pool size under load would characterise the relationship between concurrency, pool utilisation, and tail latency.

Adoption of the Clingo-based eFLINT reasoner. During the course of this thesis, a new Clingo-based implementation of the eFLINT reasoner was released [2]. The three-layer specification architecture already separates the eFLINT specification (Layers 1 and 2, which change infrequently) from the per-request scenario (Layer 3, which changes on every request approval or agreement update). This separation is well-suited to the Clingo approach: the specification can be compiled once into an answer-set program, and Clingo can then be used as an answer-set solver to evaluate individual scenarios against the compiled specification. The request approval workflow can be validated against this solved set, improving both performance and predictability of behaviour. A logical next step is adopting the Clingo-based implementation within the existing Reasoner Connector adapter, which would require replacing the `eflint-server` process pool with a Clingo-based evaluation back-end while preserving the same query interface. The compiled-specification approach is expected to reduce per-request reasoning time significantly compared to the current interpreted evaluation [2, 9].

Property-based testing. Extending the evaluation with property-based testing (generating random valid request contexts and checking invariants over the response) would increase confidence in correctness beyond the hand-crafted scenario set and could uncover edge cases in the predicate projection logic.

Active obligation enforcement. Implementing the Obligation Manager to monitor duty deadlines, detect violations, and trigger compensatory actions (e.g., revoking sessions when `obligated-log` is not fulfilled within a time bound) would complete the enforcement lifecycle.

Attribute-change monitoring. Once DYNAMOS exposes attribute-monitoring hooks, the `permitted-continuation` mechanism can be extended to trigger re-evaluation on changes to subject or resource attributes, achieving the stronger form of continuous authorisation.

Distributed enforcement. Exploring a hybrid topology combining centralised agreement management with per-site enforcement (as in BRANE) would address consortia that require local sovereignty while retaining the centralised policy-update benefits demonstrated in this thesis.

Chapter 9

Conclusion

Cross-organisational data sharing through Digital Data Marketplaces requires policy enforcement that is both formally grounded and adaptive to runtime change. This thesis examined how such enforcement can be integrated into DYNAMOS, a microservice-based DDM middleware whose existing policy enforcer was a mock that intersected JSON fields in etcd, lacked normative reasoning, supported only pre-access decisions, and conflated responsibilities between the orchestrator and the enforcer (chapter 4).

To address these shortcomings, the thesis followed a Design Science methodology, decomposing the main research question into three sub-questions: architectural integration (SRQ1), dynamic agreement updates (SRQ2), and the correctness of the resulting integration (SRQ3).

SRQ1 — Architectural Integration. The proposed Policy Enforcer (chapter 5) decomposes into six internal components (Request Context Builder, Policy Repository, Reasoner Connector, Session Manager, Obligation Manager, and Audit Logger), together with an embedded eFLINT Policy Reasoner running as a pooled binary process. The decomposition maps each internal component to an XACML or UCON+ role, correcting the violation identified in chapter 4: policy logic and session re-evaluation on agreement changes now reside in the Policy Enforcer (PDP) rather than in the orchestrator (CH). The Reasoner Connector acts as a two-directional modular adapter, allowing both the policy store back-end and the reasoning engine to be replaced independently.

SRQ2 — Dynamic Agreement Updates. A three-layer eFLINT specification comprising interface policy, agreement rules, and request facts (chapter 6) provides a stable query schema while allowing the agreement policy to be replaced at runtime. The runtime update mechanism makes the Policy Enforcer the authoritative driver: it validates the new policy via a synthetic probe scenario, identifies affected sessions through the Session Manager, re-evaluates each session against the new policy using the `permitted-continuation` query fact, and produces a pre-classified batch of per-session decisions for the Context Handler to act on. Policy changes can result in three distinct effects on in-flight executions: no-op, recomposition, or revocation.

SRQ3 — Correctness of the Integration. The evaluation (chapter 7) used differential testing against the eFLINT reasoner as oracle, targeting the request-approval orientation. Across seven scenarios covering full approval, partial approval, and several denial classes, the Policy Enforcer produced `ValidationResponse` messages structurally equivalent to those projected from the oracle, while additionally backing each decision with a formal derivation tree. The remaining three orientations (continuous authorisation, dynamic policy updates, and audit records) are identified as future work.

Answer to the main research question. A policy enforcer can be integrated into a microservice-based DDM middleware to ensure compliance with mutable policies by (i) replacing the procedural enforcer with a component that delegates decisions to a normative reasoner over a stable query interface, (ii) layering the policy specification so that runtime-mutable agreements can be swapped without touching enforcer code, (iii) making the enforcer itself the authoritative driver of session re-evaluation when policies change, and (iv) recording derivation trees alongside decisions to satisfy auditability. The DYNAMOS prototype demonstrates that this integration is feasible without breaking existing public interfaces and that its decisions are correct relative to the normative specification.

This thesis contributes:

1. An analysis of the DYNAMOS policy enforcement surface mapped onto the XACML data-flow architecture, identifying three concrete shortcomings (absence of normative reasoning, pre-access enforcement only, separation-of-concerns violation) that motivate redesign.
2. An architectural design for a normative policy enforcer integrated into microservice-based DDM middleware, combining XACML role conformance with UCON+ continuous authorisation and a modular adapter for the underlying reasoner and policy store.
3. A three-layer eFLINT specification pattern (interface policy, agreement rules, request facts) adapted from brane-chk to DYNAMOS, with a complete worked translation of the existing procedural validation logic into declarative normative derivations.
4. A runtime policy update mechanism in which the Policy Enforcer drives validation, session identification, and per-session re-evaluation, addressing the separation-of-concerns issue identified in the original DYNAMOS design.
5. A proof-of-concept implementation integrated into DYNAMOS, together with a differential test harness that uses the eFLINT reasoner as oracle to validate the integration plumbing.

Appendix A

eFLINT Specification Source

This appendix contains the full source of the eFLINT specification layers discussed in section 6.3.

A.1 Layer 1: Interface Policy

```
// ─────────────────────────────────────────────────────────────────────────────
// DYNAMOS Policy Enforcer — Layer 1: Interface Policy
// ─────────────────────────────────────────────────────────────────────────────
//
// Stable schema shared between the enforcer codebase and every agreement
// policy (Layer 2).  Declares base types, query facts, the request
// lifecycle (Act + Duty), and their arities.  No derivation rules.
// The agreement policy supplies Holds-when clauses via Extend Fact.
//
// Changes to this file require a codebase release.  The agreement policy can
// be replaced at runtime without touching this layer.
//
// Request-approval input
// ──────────────────────
// The only inputs from a RequestApproval message are:
//   • the requester (user)
//   • an array of data stewards (requested-steward instances)
//
// The policy enforcer fetches each steward's agreement from etcd, checks
// whether the requester has a relation, and discovers the valid archetypes
// and compute providers per steward.  At least one steward must have a
// valid relation for the request to be admissible.
// ─────────────────────────────────────────────────────────────────────────────


// ═════════════════════════════════════════════════════════════════════════════
// Base fact types
```

```
28 // ============================================================================
29
30 Fact requester
31 Fact data-steward
32 Fact dataset
33 Fact request-type
34 Fact archetype
35 Fact compute-provider
36
37 // Marks a data steward as being addressed by the current RequestApproval.
38 // Populated per request by the Request Context Builder.
39 Fact requested-steward Identified by data-steward
40
41
42 // ============================================================================
43 // Query facts — invoked by the DYNAMOS runtime
44 // ============================================================================
45
46 // Overall request admissibility: at least one requested steward must have
47 // a valid relation with the requester.
48 Fact permitted-request
49   Identified by requester
50
51 // Per-steward admissibility: the requester has standing at this steward.
52 Fact permitted-at-steward
53   Identified by requester * data-steward
54
55 // Generative queries: enumerate the archetypes and compute providers that
56 // are valid for a (requester, steward) pair — used by the orchestrator to
57 // compose the microservice chain.
58 Fact valid-archetype
59   Identified by requester * data-steward * archetype
60
61 Fact valid-compute-provider
62   Identified by requester * data-steward * compute-provider
63
64 // Ongoing decision: evaluated during continuous authorisation for an
65 // in-flight session that uses a specific archetype at a steward.
66 // A compute-provider check (valid-compute-provider) is only required
67 // when the archetype is dataThroughTtp; the runtime handles that separately.
68 Fact permitted-continuation
69   Identified by requester * data-steward * archetype
70
71 // ============================================================================
72 // Request lifecycle
73 // ============================================================================
```

```
74
75  // Audit obligation: the data steward must log the processing of a
76  // requester's data.  Active enforcement is deferred to a future
77  // Obligation Manager (design decision D5).
78  Duty obligated-log
79    Holder   data-steward
80    Claimant requester
81
82  // Records that a data request has been accepted for a (requester, steward)
83  // pair.  Created as an effect of submit-data-request.
84  Fact data-request
85    Identified by requester * data-steward
86
87  // The act that transitions a request into an accepted data-request.
88  // Its enablement conditions reference Layer-1 query facts whose derivation
89  // rules are supplied by the agreement policy (Layer 2) via Extend Fact.
90  Act submit-data-request
91    Actor requester
92    Related to data-steward
93    Holds when requested-steward(data-steward)
94      && permitted-at-steward(requester, data-steward)
95    Creates data-request(requester, data-steward)
96    Creates obligated-log(data-steward, requester)
97
```

Listing 28: Full source of `01_interface_policy.eflint` (Layer 1).

A.2 Layer 2a: Shared Agreement Rules

```
1  // ─────────────────────────────────────────────────────────────────────────
2  // DYNAMOS Policy Enforcer — Layer 2 (shared): Agreement Rules
3  // ─────────────────────────────────────────────────────────────────────────
4  //
5  // Loaded after 01_interface_policy.eflint.  Contains:
6  //
7  //   (i)   Intermediate fact declarations (agreement + relation structure)
8  //   (ii)  Extend rules that make the Layer-1 query facts derivable
9  //
10 // At runtime the Request Context Builder and PIP populate domain instances,
11 // agreement data, and request context.  For testing, each scenario file
12 // bundles everything needed after Layer 1 + Layer 2.
13 //
14 // Load order:  01_interface_policy  →  02_agreement_rules  →  scenario_*
15 // ─────────────────────────────────────────────────────────────────────────
16
17
```

```
// ============================================================================
// (i)  Intermediate facts — agreement structure
// ============================================================================

Fact agreement Identified by data-steward

Fact steward-supports-archetype Identified by data-steward * archetype
  Conditioned by agreement(data-steward)

Fact steward-supports-compute-provider Identified by data-steward * compute-provider
  Conditioned by agreement(data-steward)


// ============================================================================
// (i)  Intermediate facts — relation structure
// ============================================================================

Fact has-relation Identified by requester * data-steward
  Conditioned by agreement(data-steward)

Fact relation-allows-request-type Identified by requester * data-steward * request-type
  Conditioned by has-relation(requester, data-steward)

Fact relation-allows-dataset Identified by requester * data-steward * dataset
  Conditioned by has-relation(requester, data-steward)

Fact relation-allows-archetype Identified by requester * data-steward * archetype
  Conditioned by has-relation(requester, data-steward)

Fact relation-allows-compute-provider Identified by requester * data-steward * compute-p
  Conditioned by has-relation(requester, data-steward)


// ============================================================================
// (ii)  Derivation rules for Layer-1 query facts
// ============================================================================

Extend Fact permitted-at-steward
  Holds when agreement(data-steward)
    && has-relation(requester, data-steward)

Extend Fact permitted-request
  Holds when requested-steward(data-steward)
    && permitted-at-steward(requester, data-steward)

Extend Fact valid-archetype
```

```
  Holds when relation-allows-archetype(requester, data-steward, archetype)
    && steward-supports-archetype(data-steward, archetype)

Extend Fact valid-compute-provider
  Holds when relation-allows-compute-provider(requester, data-steward, compute-provider)
    && steward-supports-compute-provider(data-steward, compute-provider)

// Compute-provider validation is only relevant for DataThroughTtp,
// where data is routed through a third party.  For ComputeToData the
// computation travels to the data steward's own infrastructure, so no
// external compute provider is involved.  The runtime checks
// valid-compute-provider separately when the archetype requires it.
Extend Fact permitted-continuation
  Holds when permitted-at-steward(requester, data-steward)
    && valid-archetype(requester, data-steward, archetype)


```

Listing 29: Full source of `02_agreement_rules.eflint` (Layer 2a). Shared across all data stewards.

A.3 Layer 2b: Per-Steward Agreements

VU Agreement

```
// ─────────────────────────────────────────────────────────────────────────────
// DYNAMOS Policy Enforcer — Agreement: VU
// ─────────────────────────────────────────────────────────────────────────────
//
// Corresponds to the etcd entry at /policyEnforcer/agreements/VU.
//
// Requires: 01_interface_policy.eflint + 02_agreement_rules.eflint
// ─────────────────────────────────────────────────────────────────────────────

// Layer 2b — Per-steward agreement for VU (agreement_VU.eflint)

// Type instances referenced by this agreement.
+data-steward(VU).
+archetype(ComputeToData).
+archetype(DataThroughTtp).
+compute-provider(SURF).
+dataset(WageGap).
+request-type(SqlDataRequest).

+agreement(VU).

// Archetypes supported at the agreement level.
```

```
+steward-supports-archetype(VU, ComputeToData).
+steward-supports-archetype(VU, DataThroughTtp).

// Compute providers accepted by VU.
+steward-supports-compute-provider(VU, SURF).

// ── Relation: Niels ──────────────────────────────────────────────
+has-relation(Niels, VU).
+relation-allows-request-type(Niels, VU, SqlDataRequest).
+relation-allows-dataset(Niels, VU, WageGap).
+relation-allows-archetype(Niels, VU, ComputeToData).
+relation-allows-compute-provider(Niels, VU, SURF).
```

Listing 30: Full source of `agreement_VU.eflint` (Layer 2b). Per-steward agreement for VU.

UVA Agreement

```
// ─────────────────────────────────────────────────────────────────
// DYNAMOS Policy Enforcer — Agreement: UVA
// ─────────────────────────────────────────────────────────────────
//
// Corresponds to the etcd entry at /policyEnforcer/agreements/UVA.
//
// Requires: 01_interface_policy.eflint + 02_agreement_rules.eflint
// ─────────────────────────────────────────────────────────────────
// Layer 2b — Per-steward agreement for UVA (agreement_UVA.eflint)

// Type instances referenced by this agreement.
+data-steward(UVA).
+archetype(ComputeToData).
+archetype(DataThroughTtp).
+archetype(ReproducableScience).
+compute-provider(SURF).
+dataset(WageGap).
+request-type(SqlDataRequest).
+request-type(GenericRequest).

+agreement(UVA).

// Archetypes supported at the agreement level.
+steward-supports-archetype(UVA, ComputeToData).
+steward-supports-archetype(UVA, DataThroughTtp).
+steward-supports-archetype(UVA, ReproducableScience).

// Compute providers accepted by UVA.
+steward-supports-compute-provider(UVA, SURF).
```

```
30
31 // ── Relation: Niels ──────────────────────────────────────────────────
32 +has-relation(Niels, UVA).
33 +relation-allows-request-type(Niels, UVA, SqlDataRequest).
34 +relation-allows-request-type(Niels, UVA, GenericRequest).
35 +relation-allows-dataset(Niels, UVA, WageGap).
36 +relation-allows-archetype(Niels, UVA, ComputeToData).
37 +relation-allows-archetype(Niels, UVA, DataThroughTtp).
38 +relation-allows-compute-provider(Niels, UVA, SURF).
39
40 // ── Relation: Alice ──────────────────────────────────────────────────
41 +has-relation(Alice, UVA).
42 +relation-allows-request-type(Alice, UVA, SqlDataRequest).
43 +relation-allows-dataset(Alice, UVA, WageGap).
44 +relation-allows-archetype(Alice, UVA, DataThroughTtp).
45 +relation-allows-compute-provider(Alice, UVA, SURF).
```

Listing 31: Full source of `agreement_UVA.eflint` (Layer 2b). Per-steward agreement for UVA.

Appendix **B**

Harness Run Results

This appendix reproduces the results of the final harness run (`./harness run`) against the deployed prototype. For each request-approval scenario evaluated in section 7.2 one section presents: the eFLINT artefacts seeded into the DYNAMOS etcd instance, the oracle projection, the SUT response, and a per-field comparison table. All seven scenarios produced a pass verdict.
The etcd keys listed in the seeded-artefacts tables map to the following source files in the evaluation repository:

etcd key	Source file
`/policyEnforcer/eflintRules/shared`	`eflint/02_agreement_rules.eflint`
`/policyEnforcer/eflintModels/VU`	`eflint/agreement_VU.eflint`
`/policyEnforcer/eflintModels/UVA`	`eflint/agreement_UVA.eflint`
`/policyEnforcer/configs/VU`	Steward configuration (etcd-native, no `.eflint`)
`/policyEnforcer/configs/UVA`	Steward configuration (etcd-native, no `.eflint`)

B.1 S1 – Full Approval: Niels at VU and UVA

Seeded eFLINT Artefacts

Scenario file: `eflint/scenario_01_approved.eflint`. Seeded keys: `eflintRules/shared`, `eflintModels/VU`, `eflintModels/UVA`, `configs/VU`, `configs/UVA`.

Oracle Response


```
{
  "request_approved": true,
  "user": { "id": "1", "user_name": "Niels" },
  "valid_dataproviders": {
    "VU":  { "archetypes": ["ComputeToData"],
             "compute_providers": ["SURF"] },
    "UVA": { "archetypes": ["ComputeToData", "DataThroughTtp"],
             "compute_providers": ["SURF"] }
```

```
  },
  "invalid_dataproviders": []
}
```


SUT Response


```
{
  "type": "validationResponse",
  "request_type": "requestApproval",
  "request_approved": true,
  "user": { "id": "1", "user_name": "Niels" },
  "valid_dataproviders": {
    "UVA": { "archetypes": ["ComputeToData", "DataThroughTtp"],
             "compute_providers": ["SURF"] },
    "VU":  { "archetypes": ["ComputeToData"],
             "compute_providers": ["SURF"] }
  },
  "invalid_dataproviders": [],
  "auth": { "access_token": "1234", "refresh_token": "1234" },
  "valid_archetypes": {
    "user_name": "Niels",
    "archetypes": {
      "UVA": { "archetypes": ["ComputeToData", "DataThroughTtp"] },
      "VU":  { "archetypes": ["ComputeToData"] }
    }
  }
}
```


Field Comparison

Field	Expected (oracle)	Actual (SUT)	Rule
request_approved	true	true	bit-equal
invalid_dataproviders	[]	[]	set equality
valid_dataproviders keys	{UVA, VU}	{UVA, VU}	set equality
auth present	non-empty when approved	present	token values masked
UVA: archetypes	[ComputeToData, DataThroughTtp]	[ComputeToData, DataThroughTtp]	set equality
UVA: compute_providers	[SURF]	[SURF]	set equality
VU: archetypes	[ComputeToData]	[ComputeToData]	set equality
VU: compute_providers	[SURF]	[SURF]	set equality

B.2 S2a – Partial Approval: Alice at UVA Only

Seeded eFLINT Artefacts

Scenario file: `eflint/scenario_02a_approved_alice_uva.eflint`. Seeded keys: `eflintRules/shared`, `eflintModels/UVA`, `configs/UVA`.

Oracle Response


```
{
  "request_approved": true,
  "user": { "id": "2", "user_name": "Alice" },
  "valid_dataproviders": {
    "UVA": { "archetypes": ["DataThroughTtp"],
             "compute_providers": ["SURF"] }
  },
  "invalid_dataproviders": []
}
```


SUT Response


```
{
  "type": "validationResponse",
  "request_type": "requestApproval",
  "request_approved": true,
  "user": { "id": "2", "user_name": "Alice" },
  "valid_dataproviders": {
    "UVA": { "archetypes": ["DataThroughTtp"],
             "compute_providers": ["SURF"] }
  },
  "invalid_dataproviders": [],
  "auth": { "access_token": "1234", "refresh_token": "1234" },
  "valid_archetypes": {
    "user_name": "Alice",
    "archetypes": {
      "UVA": { "archetypes": ["DataThroughTtp"] }
    }
  }
}
```


Field Comparison

Field	Expected (oracle)	Actual (SUT)	Rule
request_approved	true	true	bit-equal
invalid_dataproviders	[]	[]	set equality
valid_dataproviders keys	{UVA}	{UVA}	set equality
auth present	non-empty when approved	present	token values masked
UVA: archetypes	[DataThroughTtp]	[DataThroughTtp]	set equality
UVA: compute_providers	[SURF]	[SURF]	set equality

B.3 S2b – Partial Approval with Partial Denial: Alice at VU and UVA

Seeded eFLINT Artefacts

Scenario file: `eflint/scenario_02b_approved_alice_partial.eflint`. Seeded keys: `eflintRules/shared`, `eflintModels/VU`, `eflintModels/UVA`, `configs/VU`, `configs/UVA`.

Oracle Response


```
{
  "request_approved": true,
  "user": { "id": "2", "user_name": "Alice" },
  "valid_dataproviders": {
    "UVA": { "archetypes": ["DataThroughTtp"],
             "compute_providers": ["SURF"] }
  },
  "invalid_dataproviders": ["VU"]
}
```


SUT Response


```
{
  "type": "validationResponse",
  "request_type": "requestApproval",
  "request_approved": true,
  "user": { "id": "2", "user_name": "Alice" },
  "valid_dataproviders": {
    "UVA": { "archetypes": ["DataThroughTtp"],
             "compute_providers": ["SURF"] }
  },
  "invalid_dataproviders": ["VU"],
  "auth": { "access_token": "1234", "refresh_token": "1234" },
  "valid_archetypes": {
    "user_name": "Alice",
    "archetypes": {
      "UVA": { "archetypes": ["DataThroughTtp"] }
    }
  }
}
```


Field Comparison

Field	Expected (oracle)	Actual (SUT)	Rule
request_approved	true	true	bit-equal
invalid_dataproviders	[VU]	[VU]	set equality
valid_dataproviders keys	{UVA}	{UVA}	set equality
auth present	non-empty when approved	present	token values masked
UVA: archetypes	[DataThroughTtp]	[DataThroughTtp]	set equality
UVA: compute_providers	[SURF]	[SURF]	set equality

B.4 S3a – Denial: No Agreement (Bob at SURF_org)

Seeded eFLINT Artefacts

Scenario file: `eflint/scenario_03a_denied_bob_no_agreement.eflint`. Seeded keys: `eflintRules/shared` only; no steward-specific model or configuration exists for `SURF_org`.

Oracle Response


```
{
  "request_approved": false,
  "user": { "id": "3", "user_name": "Bob" },
  "valid_dataproviders": {},
  "invalid_dataproviders": ["SURF_org"]
}
```


SUT Response


```
{
  "type": "validationResponse",
  "request_type": "requestApproval",
  "request_approved": false,
  "user": { "id": "3", "user_name": "Bob" },
  "valid_dataproviders": {},
  "invalid_dataproviders": ["SURF_org"],
  "valid_archetypes": { "user_name": "" }
}
```


Field Comparison

Field	Expected (oracle)	Actual (SUT)	Rule
request_approved	false	false	bit-equal
invalid_dataproviders	[SURF_org]	[SURF_org]	set equality
valid_dataproviders keys	{}	{}	set equality

B.5 S3b – Denial: No Relations (Bob at VU and UVA)

Seeded eFLINT Artefacts

Scenario file: `eflint/scenario_03b_denied_bob_no_relation.eflint`. Seeded keys: `eflintRules/shared`, `eflintModels/VU`, `eflintModels/UVA`, `configs/VU`, `configs/UVA`. Models and configurations for both stewards are present; Bob lacks any qualifying data-sharing relation in those models.

Oracle Response


```
{
  "request_approved": false,
  "user": { "id": "3", "user_name": "Bob" },
  "valid_dataproviders": {},
  "invalid_dataproviders": ["UVA", "VU"]
}
```


SUT Response


```
{
  "type": "validationResponse",
```

```
  "request_type": "requestApproval",
  "request_approved": false,
  "user": { "id": "3", "user_name": "Bob" },
  "valid_dataproviders": {},
  "invalid_dataproviders": ["VU", "UVA"],
  "valid_archetypes": { "user_name": "Bob" }
}
```


Field Comparison

Field	Expected (oracle)	Actual (SUT)	Rule
request_approved	false	false	bit-equal
invalid_dataproviders	[UVA, VU]	[VU, UVA]	set equality
valid_dataproviders keys	{}	{}	set equality

B.6 S4a – Denial: Wrong Steward (Alice at VU)

Seeded eFLINT Artefacts

Scenario file: `eflint/scenario_04a_denied_alice_wrong_steward.eflint`. Seeded keys: `eflintRules/shared`, `eflintModels/VU`, `eflintModels/UVA`, `configs/VU`, `configs/UVA`. Both VU and UVA models are present in etcd, but Alice's request targets VU only; Alice holds no qualifying relation at VU.

Oracle Response


```
{
  "request_approved": false,
  "user": { "id": "2", "user_name": "Alice" },
  "valid_dataproviders": {},
  "invalid_dataproviders": ["VU"]
}
```


SUT Response


```
{
  "type": "validationResponse",
  "request_type": "requestApproval",
  "request_approved": false,
  "user": { "id": "2", "user_name": "Alice" },
  "valid_dataproviders": {},
  "invalid_dataproviders": ["VU"],
  "valid_archetypes": { "user_name": "Alice" }
}
```

Field Comparison

Field	Expected (oracle)	Actual (SUT)	Rule
request_approved	false	false	bit-equal
invalid_dataproviders	[VU]	[VU]	set equality
valid_dataproviders keys	{}	{}	set equality

B.7 S4b – Restricted Archetype: Alice at UVA (DataThroughTtp Only)

Seeded eFLINT artefacts.

Scenario file: `eflint/scenario_04b_denied_alice_restricted_archetype.eflint`. Seeded keys: `eflintRules/shared`, `eflintModels/UVA`, `configs/UVA`. Alice's agreement at UVA restricts her to the `DataThroughTtp` archetype; `ComputeToData` and `ReproducableScience` are explicitly excluded.

Oracle response.


```
{
  "request_approved": true,
  "user": { "id": "2", "user_name": "Alice" },
  "valid_dataproviders": {
    "UVA": { "archetypes": ["DataThroughTtp"],
             "compute_providers": ["SURF"] }
  },
  "invalid_dataproviders": []
}
```


SUT response.


```
{
  "type": "validationResponse",
  "request_type": "requestApproval",
  "request_approved": true,
  "user": { "id": "2", "user_name": "Alice" },
  "valid_dataproviders": {
    "UVA": { "archetypes": ["DataThroughTtp"],
             "compute_providers": ["SURF"] }
  },
  "invalid_dataproviders": [],
  "auth": { "access_token": "1234", "refresh_token": "1234" },
  "valid_archetypes": {
    "user_name": "Alice",
    "archetypes": {
      "UVA": { "archetypes": ["DataThroughTtp"] }
    }
  }
}
```

Field comparison.

Field	Expected (oracle)	Actual (SUT)	Rule
request_approved	true	true	bit-equal
invalid_dataproviders	[]	[]	set equality
valid_dataproviders keys	{UVA}	{UVA}	set equality
auth present	non-empty when approved	present	token values masked
UVA: archetypes	[DataThroughTtp]	[DataThroughTtp]	set equality
UVA: compute_providers	[SURF]	[SURF]	set equality

References

[1] P. Arcaini, E. Riccobene and P. Scandurra. "Modeling and Analyzing MAPE-k Feedback Loops for Self-Adaptation". In: *2015 IEEE/ACM 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 2015 IEEE/ACM 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems. May 2015, pp. 13–23. doi: 10.1109/SEAMS.2015.10. url: https://ieeexplore.ieee.org/abstract/document/7194653/authors (visited on 06/10/2025).

[2] L. T. van Binsbergen, C. A. Esterhuyse and T. Müller. *Reflections on the Design, Applications and Implementations of the Normative Specification Language eFLINT*. 15th Nov. 2025. doi: 10.48550/arXiv.2511.12276. arXiv: 2511.12276 [cs]. url: http://arxiv.org/abs/2511.12276 (visited on 13/02/2026). Pre-published.

[3] L. T. van Binsbergen, M. C. Steketee, M. G. Kebede, H. L. Janssen and T. M. van Engers. *Lawful and Accountable Personal Data Processing with GDPR-based Access and Usage Control in Distributed Systems*. 10th Mar. 2025. doi: 10.48550/arXiv.2503.07172. arXiv: 2503.07172 [cs]. url: http://arxiv.org/abs/2503.07172 (visited on 07/08/2025). Pre-published.

[4] *BraneFramework/Brane: Programmable Orchestration of Applications and Networking*. url: https://github.com/BraneFramework/brane/tree/main (visited on 03/05/2026).

[5] S. W. Driessen, G. Monsieur and W.-J. Van Den Heuvel. "Data Market Design: A Systematic Literature Review". In: *IEEE Access* 10 (2022), pp. 33123–33153. issn: 2169-3536. doi: 10.1109/ACCESS.2022.3161478. url: https://ieeexplore.ieee.org/document/9739681 (visited on 02/09/2025).

[6] S. Easterbrook, J. Singer, M.-A. Storey and D. Damian. "Selecting Empirical Methods for Software Engineering Research". In: doi: 10.1007/978-1-84800-044-5_11. url: https://link.springer.com/chapter/10.1007/978-1-84800-044-5_11 (visited on 21/10/2025).

[7] A. Eitel, C. Jung, R. Brandstädter, A. Hosseinzadeh, S. Bader, Ch. Kühnle, P. Birnstill, G. Brost, Gall, F. Bruckner, N. Weißenberg and B. Korth. *Usage Control in the International Data Spaces*. Version 3.0. Zenodo, 1st Mar. 2021. doi: 10.5281/ZENODO.5675884. url: https://zenodo.org/record/5675884 (visited on 08/09/2025).

[8] C. A. Esterhuyse, T. Müller, L. T. Van Binsbergen and A. S. Z. Belloum. "Exploring the Enforcement of Private, Dynamic Policies on Medical Workflow Execution". In: *2022 IEEE 18th International Conference on E-Science (e-Science)*. 2022 IEEE 18th International Conference on E-Science (e-Science). Oct. 2022, pp. 481–486. doi: 10.1109/eScience55777.2022.00086. url: https://ieeexplore.ieee.org/document/9973688 (visited on 28/05/2025).

[9] C. A. Esterhuyse, T. Müller and L. T. van Binsbergen. "A Stable Model Semantics for eFLINT Norm Specifications and Model Checking Scenarios". In: *Proceedings of the 24th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*. GPCE '25. New York, NY, USA: Association for Computing Machinery, 27th June 2025, pp. 80–93. isbn: 979-8-4007-1995-0. doi: 10.1145/3742876.3742882. url: https://dl.acm.org/doi/10.1145/3742876.3742882 (visited on 02/12/2025).

[10] "eXtensible Access Control Markup Language (XACML) Version 3.0". In: (2013).

[11] N. Farhadighalati, L. A. Estrada-Jimenez, S. Nikghadam-Hojjati and J. Barata. "A Systematic Review of Access Control Models: Background, Existing Research, and Challenges". In: *IEEE Access* 13 (2025), pp. 17777–17806. issn: 2169-3536. doi: 10.1109/ACCESS.2025.3533145. url: https://ieeexplore.ieee.org/abstract/document/10850915 (visited on 20/10/2025).

[12] C. Goumopoulos. "Smart City Middleware: A Survey and a Conceptual Framework". In: *IEEE Access* 12 (2024), pp. 4015–4047. issn: 2169-3536. doi: 10.1109/ACCESS.2023.3349376. url: https://ieeexplore.ieee.org/abstract/document/10379798 (visited on 09/09/2025).

[13] A. Hariri, A. Ibrahim, B. Alangot, S. Bandopadhyay, A. La Marra, A. Rosetti, H. Joumaa and T. Dimitrakos. "UCON+: Comprehensive Model, Architecture and Implementation for Usage Control and Continuous Authorization". In: *Collaborative Approaches for Cyber Security in Cyber-Physical Systems*. Ed. by T. Dimitrakos, J. Lopez and F. Martinelli. Cham: Springer International Publishing, 2023, pp. 209–226. isbn: 978-3-031-16088-2. doi: 10.1007/978-3-031-16088-2_10. url: https://doi.org/10.1007/978-3-031-16088-2_10 (visited on 26/03/2026).

[14] W. N. Hohfeld. "Fundamental Legal Conceptions as Applied in Judicial Reasoning". In: *The Yale Law Journal* 26.8 (1917), pp. 710–770. issn: 0044-0094. doi: 10.2307/786270. JSTOR: 786270. url: https://www.jstor.org/stable/786270 (visited on 16/04/2026).

[15] V. C. Hu, D. R. Kuhn, D. F. Ferraiolo and J. Voas. "Attribute-Based Access Control". In: *Computer* 48.2 (Feb. 2015), pp. 85–88. issn: 1558-0814. doi: 10.1109/MC.2015.33. url: https://ieeexplore.ieee.org/document/7042715 (visited on 26/03/2026).

[16] C. Jung and J. Dörr. "Data Usage Control". In: *Designing Data Spaces : The Ecosystem Approach to Competitive Advantage*. Ed. by B. Otto, M. ten Hompel and S. Wrobel. Cham: Springer International Publishing, 2022, pp. 129–146. isbn: 978-3-030-93975-5. doi: 10.1007/978-3-030-93975-5_8. url: https://doi.org/10.1007/978-3-030-93975-5_8 (visited on 10/03/2026).

[17] M. G. Kebede, G. Sileno and T. Van Engers. "A Critical Reflection on ODRL". In: *AI Approaches to the Complexity of Legal Systems XI-XII*. Ed. by V. Rodríguez-Doncel, M. Palmirani, M. Araszkiewicz, P. Casanovas, U. Pagallo and G. Sartor. Cham: Springer International Publishing, 2021, pp. 48–61. isbn: 978-3-030-89811-3. doi: 10.1007/978-3-030-89811-3_4.

[18] B. Otto, M. ten Hompel and S. Wrobel, eds. *Designing Data Spaces: The Ecosystem Approach to Competitive Advantage*. Springer Nature, 2022. isbn: 978-1-338-20122-2. doi: 10.1007/978-3-030-93975-5. url: https://library.oapen.org/handle/20.500.12657/57901 (visited on 09/09/2025).

[19] M. Palviainen and J. Suksi. "Data Marketplace Research: A Review of the State-of-the-Art with a Focus on Smart Cities and on Edge Data Exchange and Trade". In: *2023 Smart City Symposium Prague (SCSP)*. 2023 Smart City Symposium Prague (SCSP). May 2023, pp. 1–7. doi: 10.1109/SCSP58044.2023.10146227. url: https://ieeexplore.ieee.org/abstract/document/10146227/metrics (visited on 02/09/2025).

[20] A. M. Reina Quintero, S. Martínez Pérez, Á. J. Varela-Vaca, M. T. Gómez López and J. Cabot. "A Domain-Specific Language for the Specification of UCON Policies". In: *Journal of Information Security and Applications* 64 (1st Feb. 2022), p. 103006. doi: 10.1016/j.jisa.2021.103006. url: https://www.sciencedirect.com/science/article/pii/S221421262100212X.

[21] R. Sandhu, E. Coyne, H. Feinstein and C. Youman. "Role-Based Access Control Models". In: *Computer* 29.2 (Feb. 1996), pp. 38–47. issn: 1558-0814. doi: 10.1109/2.485845. url: https://ieeexplore.ieee.org/document/485845 (visited on 26/03/2026).

[22] R. Sandhu and J. Park. "Usage Control: A Vision for Next Generation Access Control". In: *Computer Network Security*. Ed. by V. Gorodetsky, L. Popyack and V. Skormin. Berlin, Heidelberg: Springer, 2003, pp. 17–31. isbn: 978-3-540-45215-7. doi: 10.1007/978-3-540-45215-7_2.

[23] S. Shakeri, V. Maccatrozzo, L. Veen, R. Bakhshi, L. Gommans, C. de Laat and P. Grosso. "Modeling and Matching Digital Data Marketplace Policies". In: *2019 15th International Conference on eScience (eScience)*. 2019 15th International Conference on eScience (eScience). Sept. 2019, pp. 570–577. doi: 10.1109/eScience.2019.00078. url: https://ieeexplore.ieee.org/abstract/document/9041754 (visited on 09/09/2025).

[24] S. Shakeri, L. Veen and P. Grosso. "Evaluation of Container Overlays for Secure Data Sharing". In: *2020 IEEE 45th LCN Symposium on Emerging Topics in Networking (LCN Symposium)*. 2020 IEEE 45th LCN Symposium on Emerging Topics in Networking (LCN Symposium). Nov. 2020, pp. 99–108. doi: 10.1109/LCNSymposium50271.2020.9363266. url: https://ieeexplore.ieee.org/document/9363266/ (visited on 07/08/2025).

[25] J. Stutterheim. "DYNAMOS: Dynamically Adaptive Microservice-Based OS". 2023.

[26] J. Stutterheim, A. Mifsud and A. Oprescu. "DYNAMOS: Dynamic Microservice Composition for Data-Exchange Systems, Lessons Learned". In: *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*. 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C). June 2024, pp. 8–15. doi: 10.1109/ICSA-C63560.2024.00008. url: https://ieeexplore.ieee.org/abstract/document/10628120 (visited on 22/05/2025).

[27] O. Valkering, R. Cushing and A. Belloum. "Brane: A Framework for Programmable Orchestration of Multi-Site Applications". In: *2021 IEEE 17th International Conference on eScience (eScience)*. 2021 IEEE 17th International Conference on eScience (eScience). Sept. 2021, pp. 277–282. doi: 10.1109/eScience51609.2021.00056. url: https://ieeexplore.ieee.org/document/9582292 (visited on 01/09/2025).

[28] L. T. van Binsbergen, M. G. Kebede, J. Baugh, T. van Engers and D. G. van Vuurden. "Dynamic Generation of Access Control Policies from Social Policies". In: *Procedia Computer Science*. 12th International Conference on Emerging Ubiquitous Systems and Pervasive Networks / 11th International Conference on Current and Future Trends of Information and Communication Technologies in Healthcare 198 (1st Jan. 2022), pp. 140–147. issn: 1877-0509. doi: 10.1016/j.procs.2021.12.221. url: https://www.sciencedirect.com/science/article/pii/S1877050921024601 (visited on 23/09/2025).

[29] L. T. van Binsbergen, L.-C. Liu, R. van Doesburg and T. van Engers. "eFLINT: A Domain-Specific Language for Executable Norm Specifications". In: *Proceedings of the 19th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*. GPCE 2020. New York, NY, USA: Association for Computing Machinery, 16th Nov. 2020, pp. 124–136. isbn: 978-1-4503-8174-1. doi: 10.1145/3425898.3426958. url: https://dl.acm.org/doi/10.1145/3425898.3426958 (visited on 28/05/2025).

[30] L. T. van Binsbergen, M. Oost-Rosengren, H. Schreijer, F. Dijkstra and T. van Dijk. *AMdEX Reference Architecture – Version 1.0.0*. Version 1.0.0. Zenodo, 1st Feb. 2024. doi: 10.5281/ZENODO.10565915. url: https://zenodo.org/doi/10.5281/zenodo.10565915 (visited on 28/05/2025).

Acronyms

ABAC	Attribute-Based Access Control
CH	Context Handler
DDM	Digital Data Marketplace
DGA	Data Governance Act
DSL	Domain-Specific Language
GDPR	General Data Protection Regulation
MAPE-K	Monitor–Analyse–Plan–Execute over shared Knowledge
ODRL	Open Digital Rights Language
PAP	Policy Administration Point
PDP	Policy Decision Point
PEP	Policy Enforcement Point
PIP	Policy Information Point
RBAC	Role-Based Access Control
UCON	Usage Control
XACML	eXtensible Access Control Markup Language